

ANN Hyperparameters

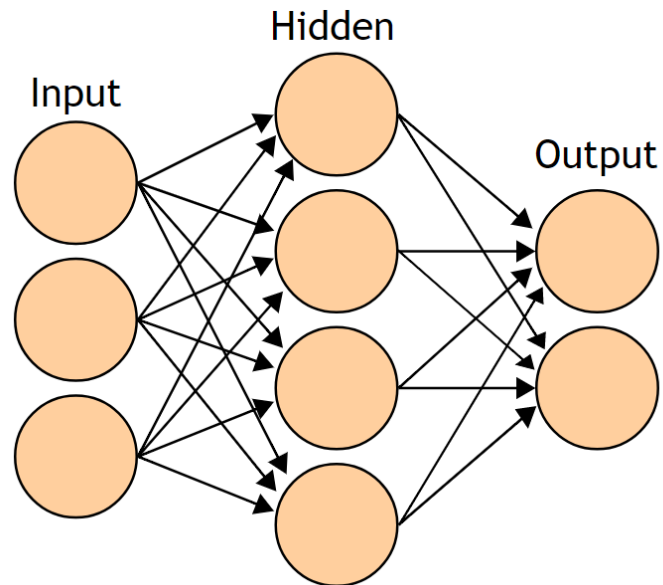
Contents

Contents

- Overfitting
- L1 and L2 Regularization
- Dropout
- Activation functions
- Learning Rate
- SGD
- Batch size

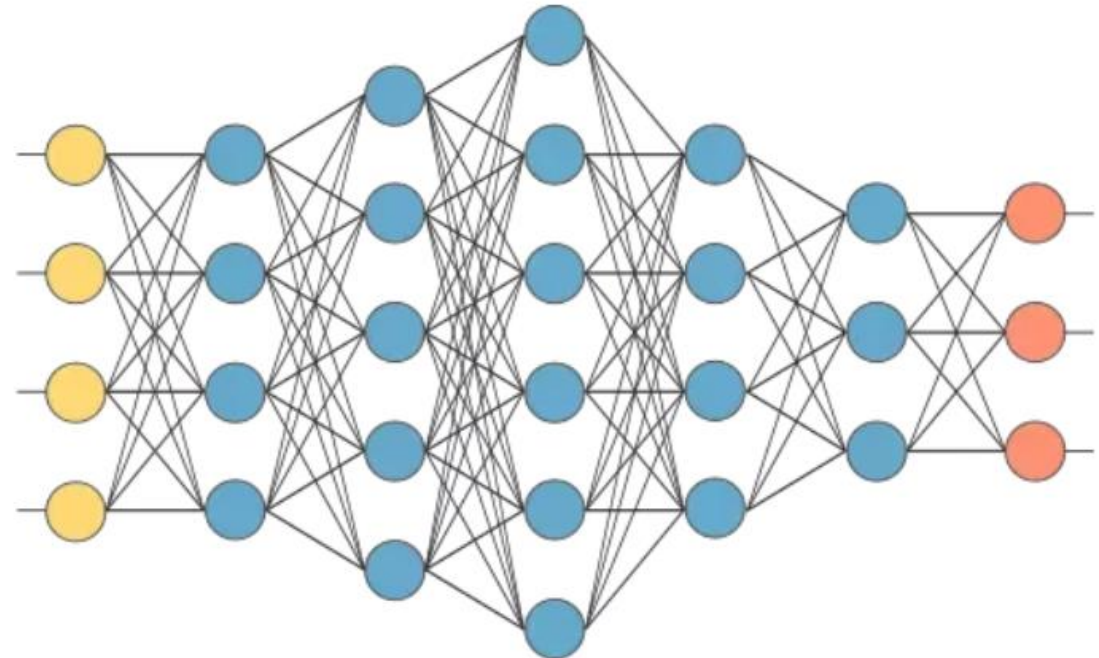
Deep vs Shallow networks

A neural network with single hidden layer is called a shallow network



shallow network

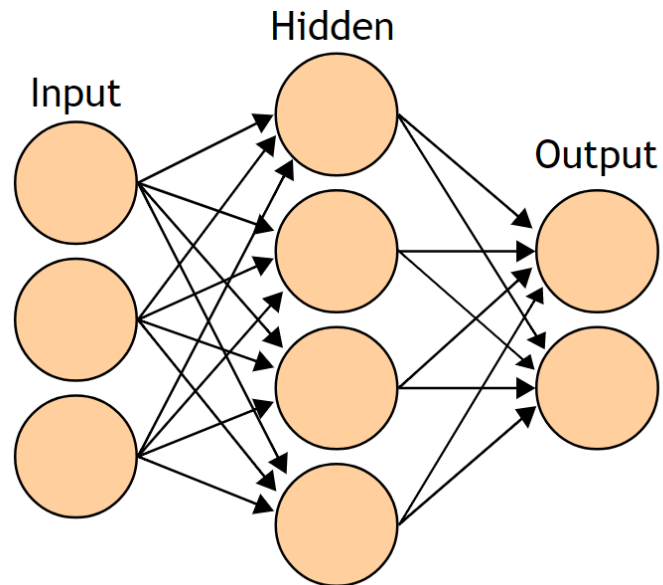
A neural network with more than one hidden layer is called deep neural network



Deep network

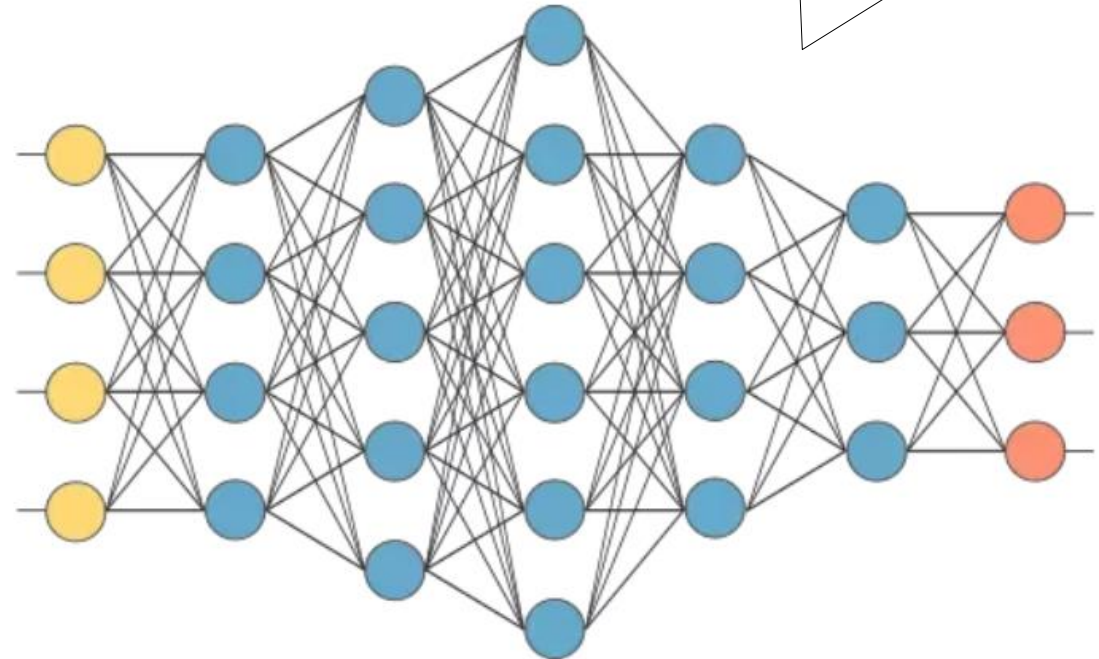
Deep vs Shallow networks

A single layer might not have the flexibility to capture all the non linear patterns in the data



shallow network

A deep network first learns the primitive features followed by high level features. This helps in building efficient models



Deep network

Deep neural networks are very powerful

- Lot of experiments have shown that a deep network with less parameters performs better than a shallow network with higher parameters
- For example deep network with hidden nodes $[10, 10, 10, 10]$ might perform better than shallow network with $[180]$ hidden nodes
- Deep neural networks are amazingly powerful.
- With sufficient number of hidden layers and nodes, we can fit a model to any type of data
- They have the power to capture any amount of non linearity

Deep neural networks - Hyper parameters

"With great power comes great responsibility"

- There are so many parameters while training
- There are some hyper parameters that we need to set at the start of the training. For eg: number of hidden layers and nodes, learning rate of weights
- There are many such hyper parameters, if we don't set them right while building the model you may end up building
 - An overfitted model
 - An under fitted model
 - Or even a never ending(non-converging) model
 - Model with vanishing gradients
- Deep learning expert need to know what are the hyper parameters and how to finetune them

Regularization

The problem of Overfitting

- Neural networks are very powerful. They have capacity to learn any type of pattern.
- With high number of hidden layers, we can fit to training data with any level of non-linearity
- Too many hidden layers might be fitting the model for random pattern or noise in the data
- Throughout the neural network algorithm we were trying optimise weights to make the error zero. This might lead to overfitting
- What is overfitting in general? Having too many polynomial terms in a regression line might result in a overfitted model.

Reduce Overfitting

- We have two options to reduce overfitting
 - Build a simple model. Drop hidden layers and nodes- But this might lead to underfitting
 - Or hidden layers and nodes but give them less weightage. This will take care of overfitting
- Instead of minimising error alone, minimise both error and weights
- The regularization term Imposes some penalty on weights. It impacts the overall weights and reduces the overfitting

$$\text{New Cost Function} = \sum_{i=1}^n \left[y_i - g \left(\sum_{k=1}^m w_k h_{ki} \right) \right]^2 + \frac{1}{2} \lambda \sum_{i=1}^n w_i^2$$

Cost function regularization

$$\text{Actual Cost function or Error} = \sum_{i=1}^n \left[y_i - g \left(\sum_{k=1}^m w_k h_{ki} \right) \right]^2$$

$$\text{New Regularized error} = \sum_{i=1}^n \left[y_i - g \left(\sum_{k=1}^m w_k h_{ki} \right) \right]^2 + \frac{1}{2} \lambda \sum_{i=1}^n w_i^2$$

- Apart from minimising error sum of squares, we are minimising error + weights sum of squares also
- The second term is imposing a penalty on weights.
- This is known as Regularization

L1 and L2 Regularization

- Till now we discussed about minimizing sum of squares of weights. This is known as L2 regularization (Also known as Ridge or Decay)

$$L2 \text{ regularization} = \sum_{i=1}^n \left[y_i - g \left(\sum_{k=1}^m w_k h_{ki} \right) \right]^2 + \frac{1}{2} \lambda \sum_{i=1}^n w_i^2$$

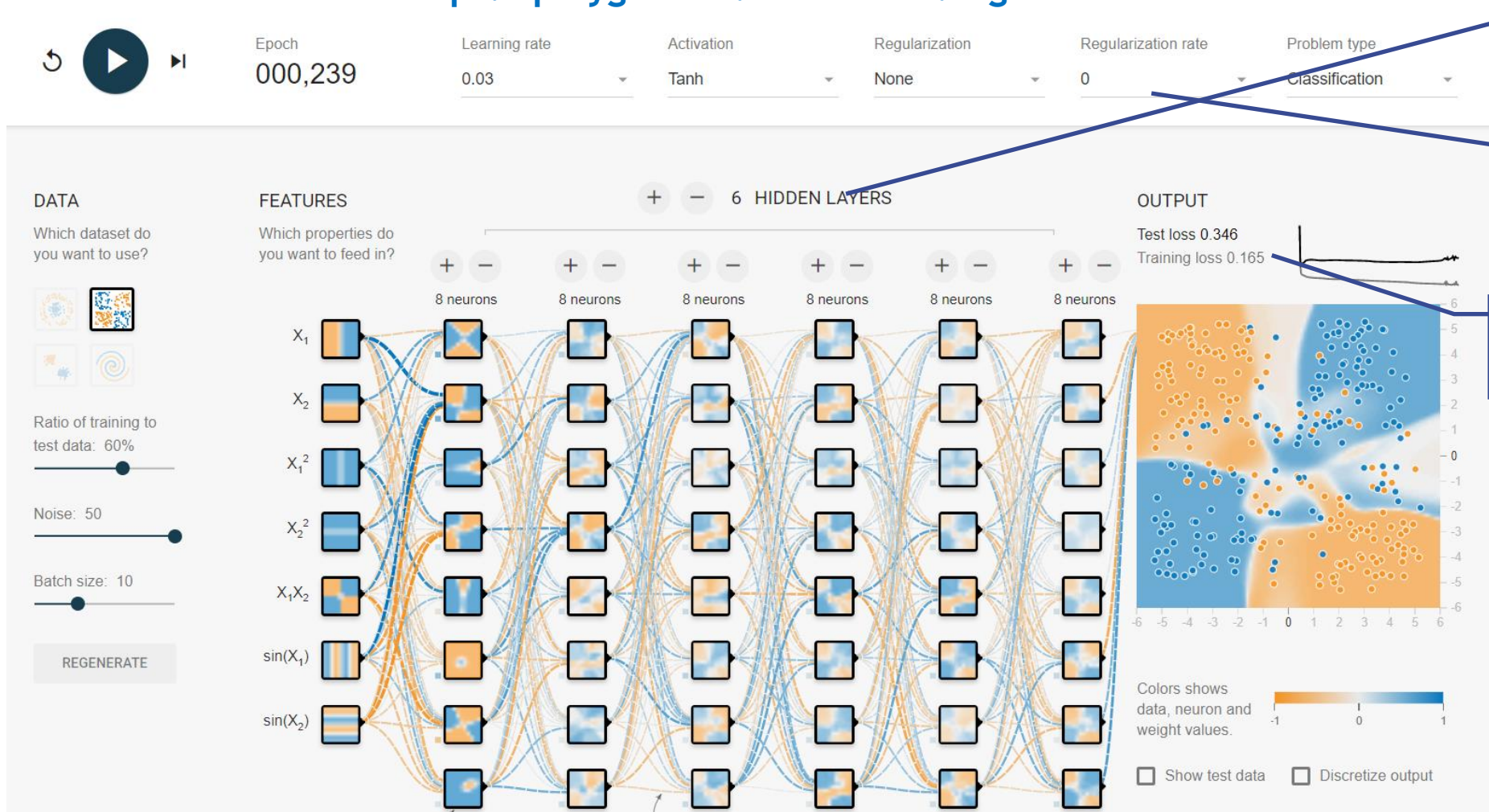
- We can minimize absolute of sum of weights; Which is known as L1 regularization (Also known as LASSO)

$$L1 \text{ regularization} = \sum_{i=1}^n \left[y_i - g \left(\sum_{k=1}^m w_k h_{ki} \right) \right]^2 + \frac{1}{2} \lambda \sum_{i=1}^n |w_i|$$

- Both of them work equally effectively. Most of the times we choose L2 regularization, for mathematical convenience

Demo: Regularization

<https://playground.tensorflow.org>

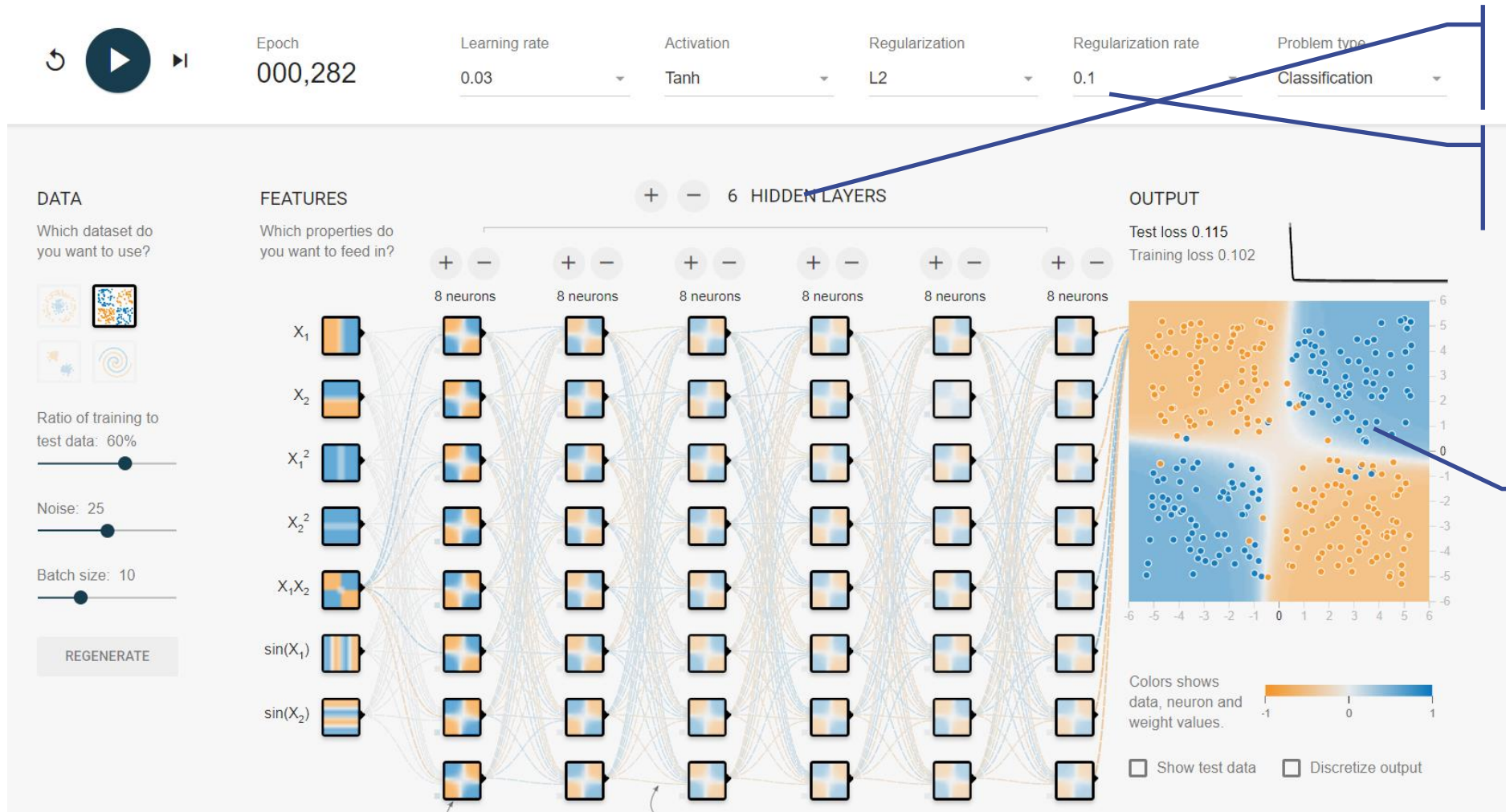


- Lot of hidden layers

- No Regularization

- Overfitted model

Demo: Regularization



- Lot of hidden layers

- With Regularization

Model is not overfitted

LAB: L1, L2 Regularization

- Try Regularization on MNIST data
- Build a model with several hidden nodes and layers. Apply regularization to avoid overfitting

Code: L1, L2 Regularization

```
from tensorflow.keras import regularizers
model_r = keras.Sequential()
model_r.add(layers.Dense(256, input_shape=(784,), kernel_regularizer=regularizers.l2(0.01)))
model_r.add(layers.Dense(128, kernel_regularizer=regularizers.l2(0.01)))
model_r.add(layers.Dense(10, activation='softmax'))
model_r.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
=====		
dense_3 (Dense)	(None, 256)	200960
dense_4 (Dense)	(None, 128)	32896
dense_5 (Dense)	(None, 10)	1290
=====		

Total params: 235,146

Trainable params: 235,146

Non-trainable params: 0

Code: L1, L2 Regularization

```
model_r.compile(loss='categorical_crossentropy', metrics=['accuracy'])
model_r.fit(x_train, y_train, epochs=10)

#Final Results
loss, acc = model_r.evaluate(x_train, y_train, verbose=2)
print("Train Accuracy: {:.5.2f}%".format(100*acc))

loss, acc = model_r.evaluate(x_test, y_test, verbose=2)
print("Test Accuracy: {:.5.2f}%".format(100*acc))
```

```
Epoch 1/10
1875/1875 [=====] - 8s 4ms/step - loss: 1.7618 - accuracy: 0.6523
Epoch 2/10
1875/1875 [=====] - 8s 4ms/step - loss: 1.3469 - accuracy: 0.7468
Epoch 3/10
1875/1875 [=====] - 8s 4ms/step - loss: 1.2432 - accuracy: 0.7713
Epoch 4/10
1875/1875 [=====] - 8s 5ms/step - loss: 1.1795 - accuracy: 0.7855
_ _ _ _ _
```

Important Note

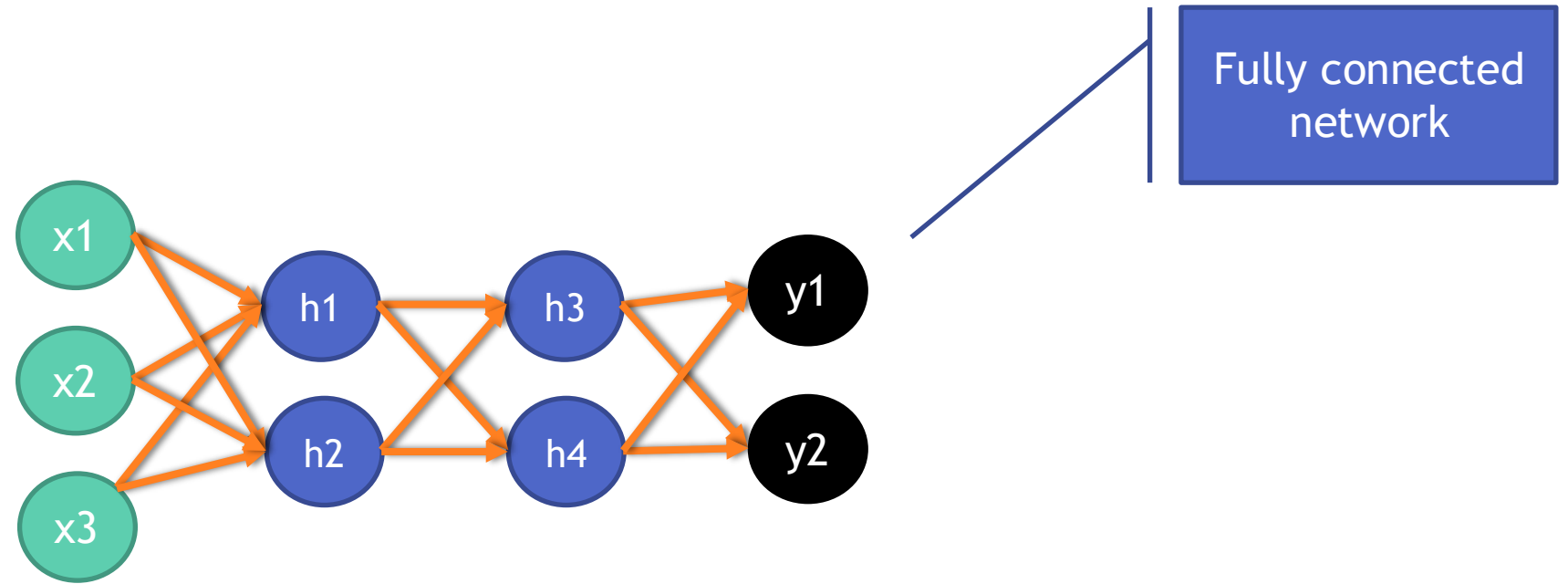
- We directly work with weights in regularization.
- It is important to standardize the data before applying regularization in neural networks
- Since a single regularization parameter is applied on all weights, its very important to bring all weights on to same scale
- Regularization parameter might not have any impact if the data is not standardised.

$$\text{New Regularized error} = \sum_{i=1}^n \left[y_i - g \left(\sum_{k=1}^m w_k h_{ki} \right) \right]^2 + \frac{1}{2} \lambda \sum_{i=1}^n w_i^2$$

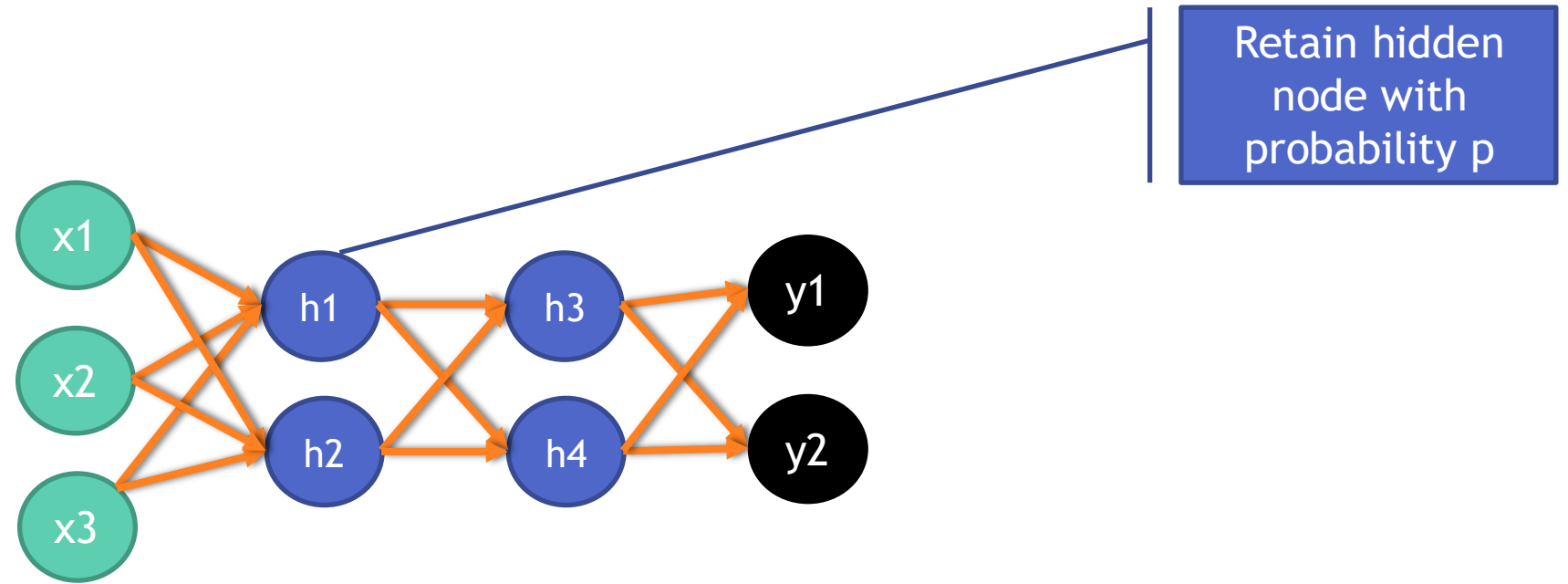
Dropout Regularization

- The previous method was about penalizing the weight with L1 and L2 regularization
 - Dropout method is also used for avoiding overfitting
 - Dropping of the hidden units while building the model
 - Ignoring few hidden units in each feed forward pass of the back propagation algorithm
-
- ‘Dropping out few hidden units in every iteration’
 - Dropout method forces each neuron to learn a few patterns, it ensures that a single neuron is NOT dominating in the entire model.

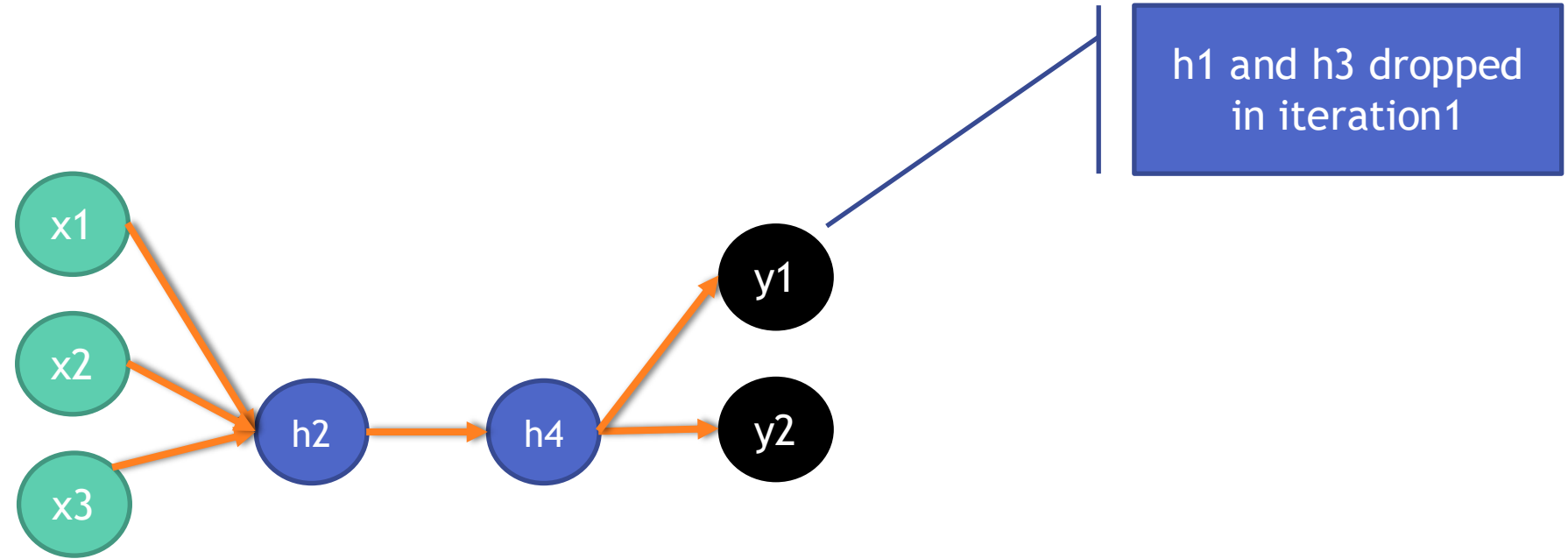
How Dropout Regularization works?



How Dropout Regularization works?

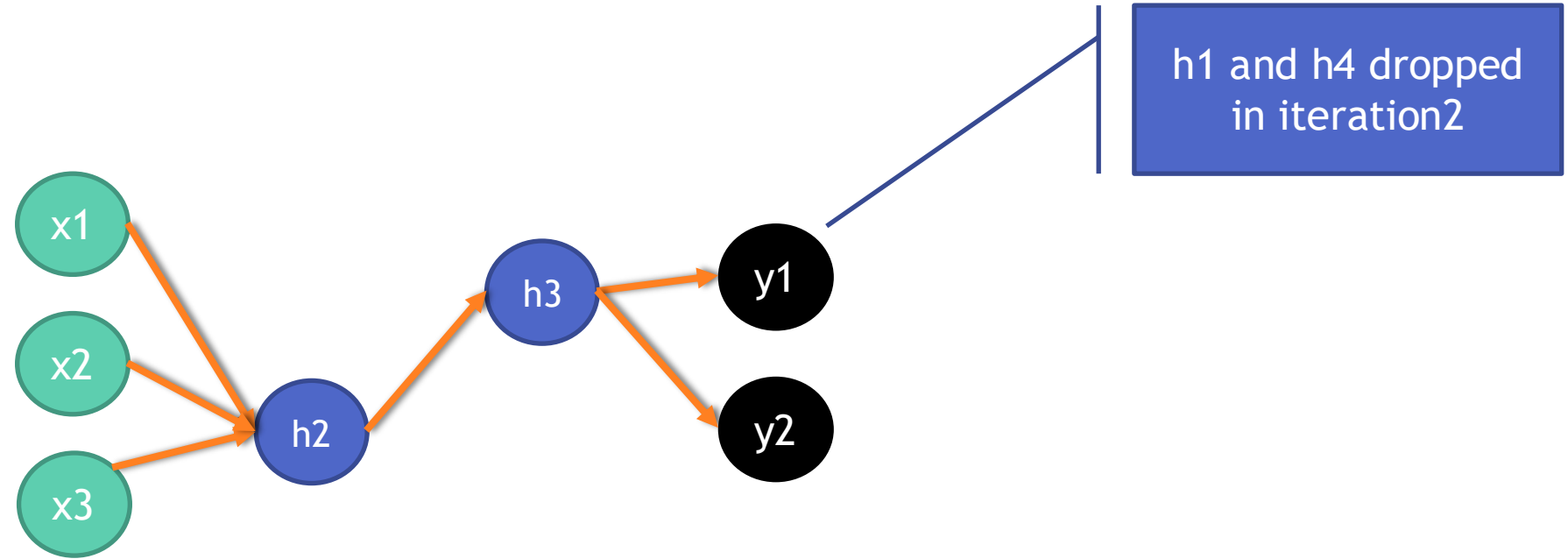


Dropout Regularization

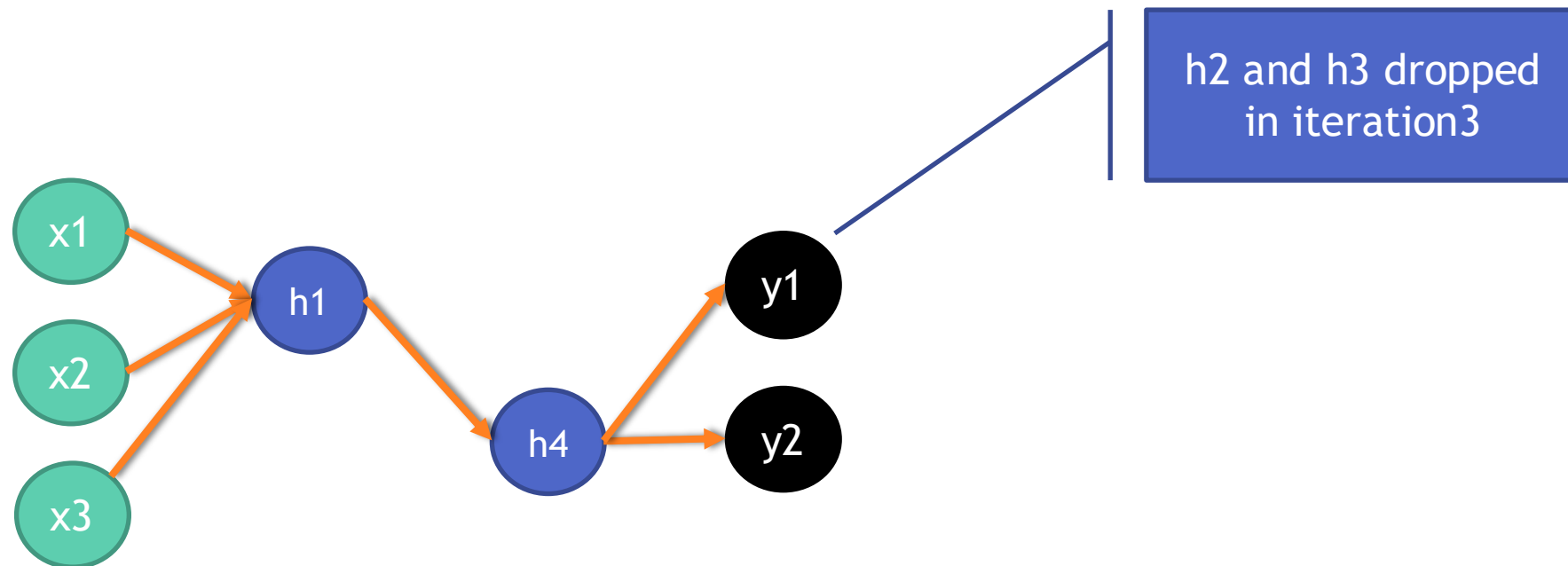


- This is called a thinned network
- In each iteration, backpropagation is performed on the thinned network

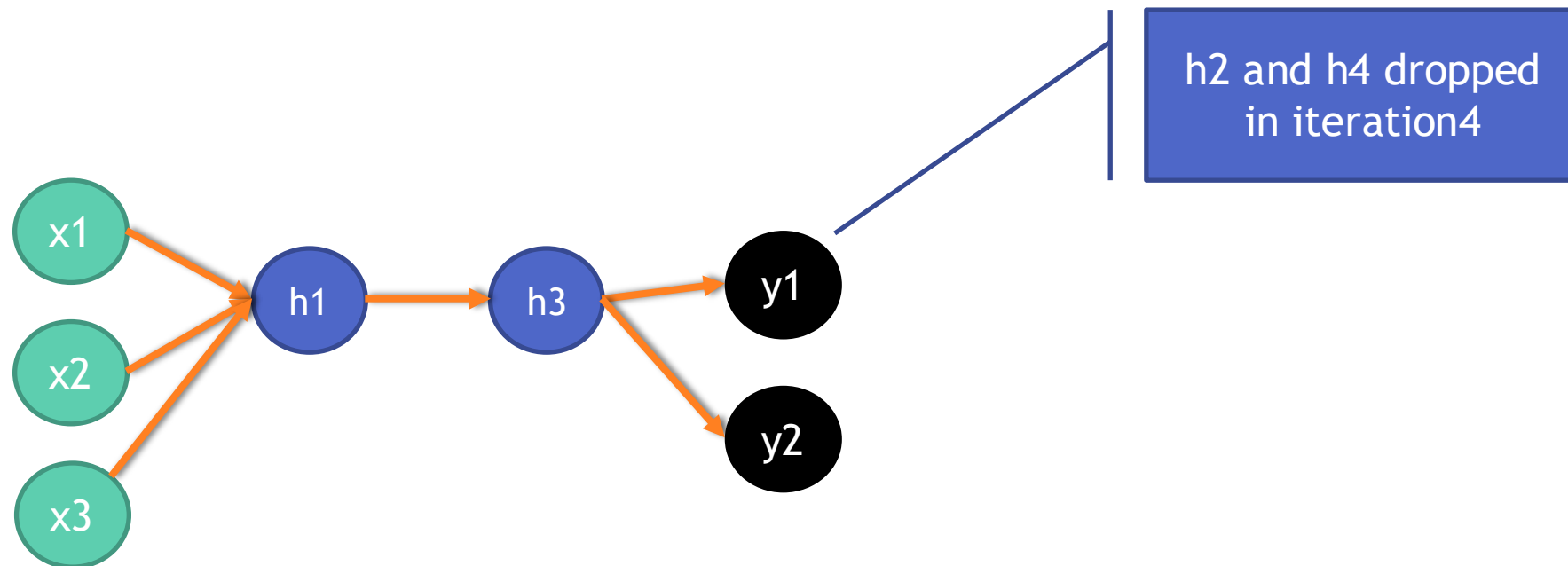
Dropout Regularization



Dropout Regularization



Dropout Regularization

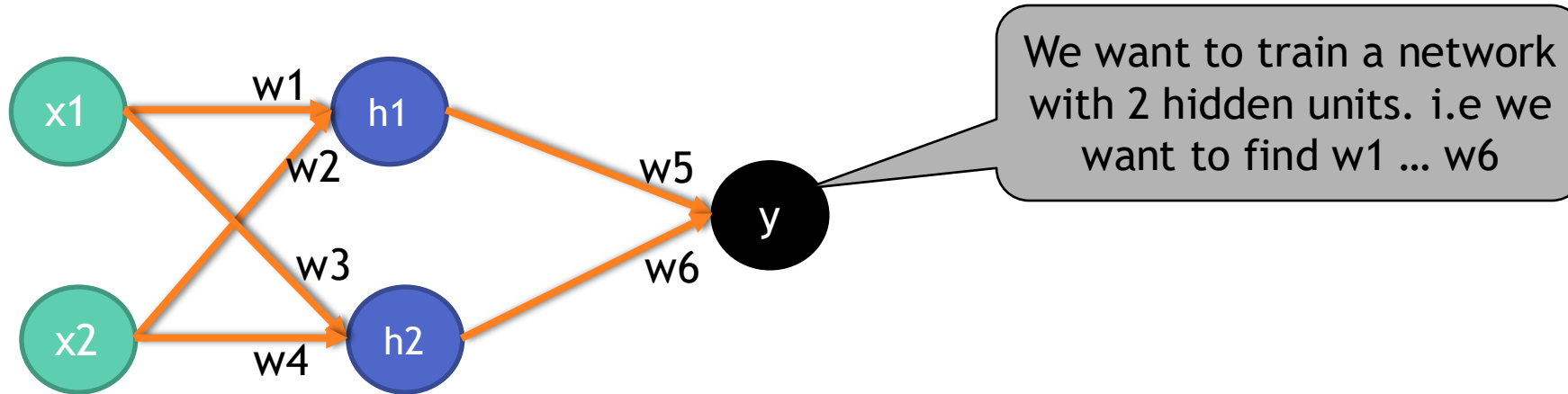


Final weights in the model(While testing)

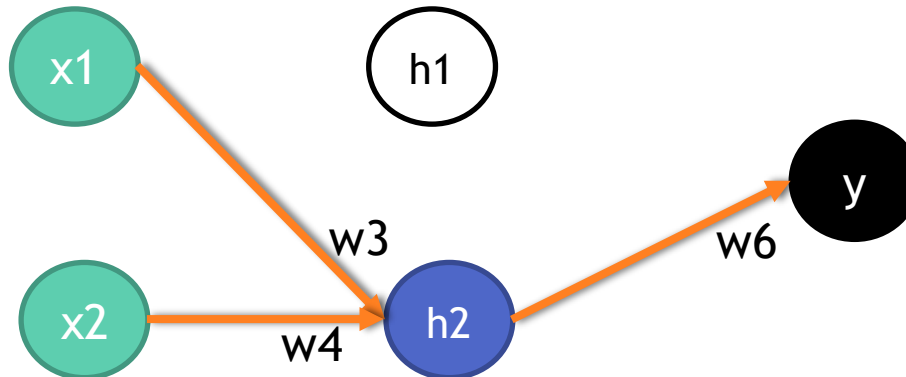
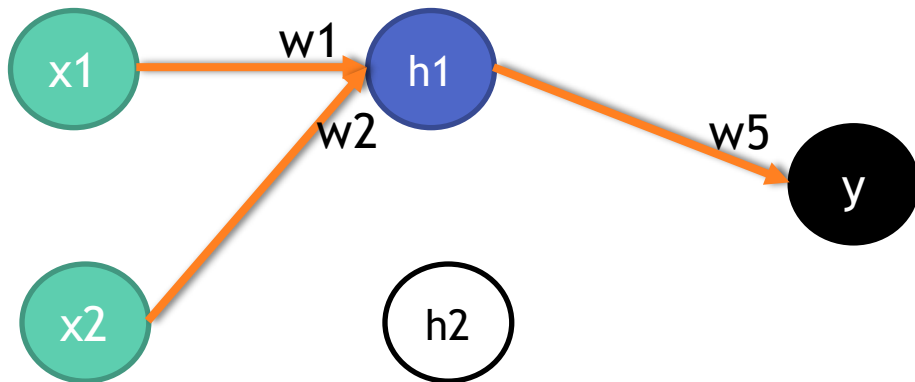
- At training phase
 - In each iteration, drop few hidden units and calculate weights
 - For example each unit is kept with probability 0.5 in each iteration
 - Train the network and take the final weights matrix from all networks.
- At the test time
 - We use all the weights, as if there was no drop out.
 - We may have to incorporate dropout impact on weights while perdition
 - If probability of keeping each node is 0.5 then multiply weights with 0.5

Lets see an example to understand this procedure

Dropout Neural network at training time



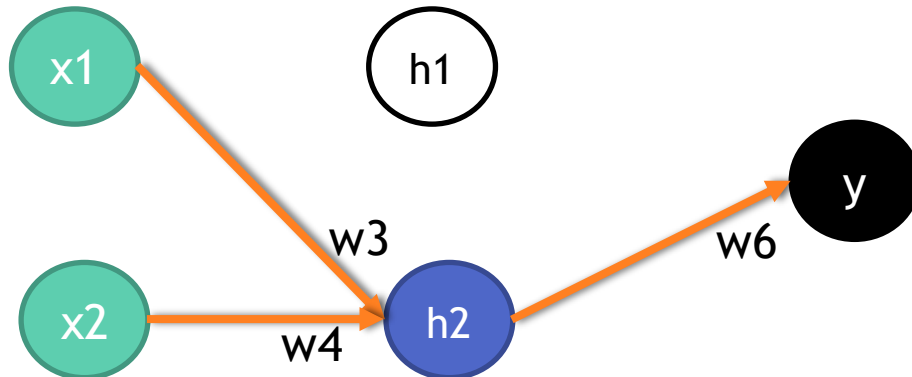
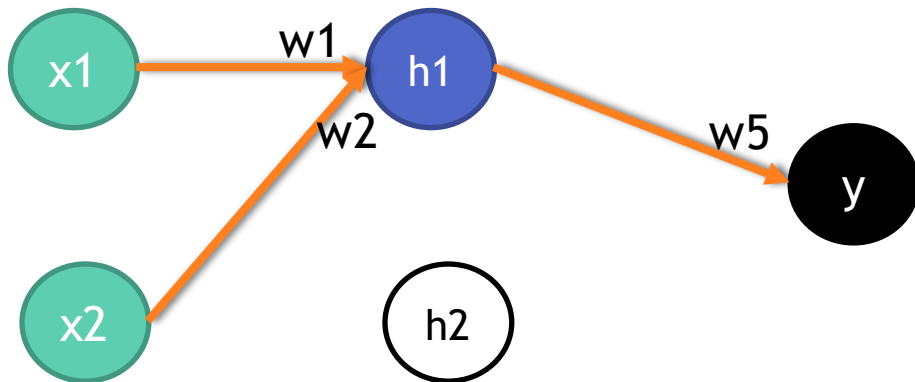
Let 0.5 be the probability of retaining a unit. We will see below structures while training



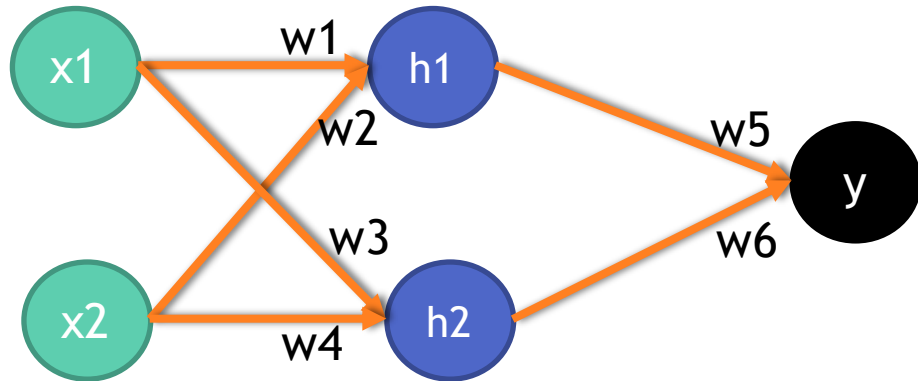
Dropout Neural network at training time

- At different iterations while training we can write the formula for y as
 - $y = \text{sigmoid}(w_5 \cdot h_1)$ or
 - $y = \text{sigmoid}(w_6 \cdot h_2)$

Let 0.5 be the probability of retaining a unit. We will see below structures while training

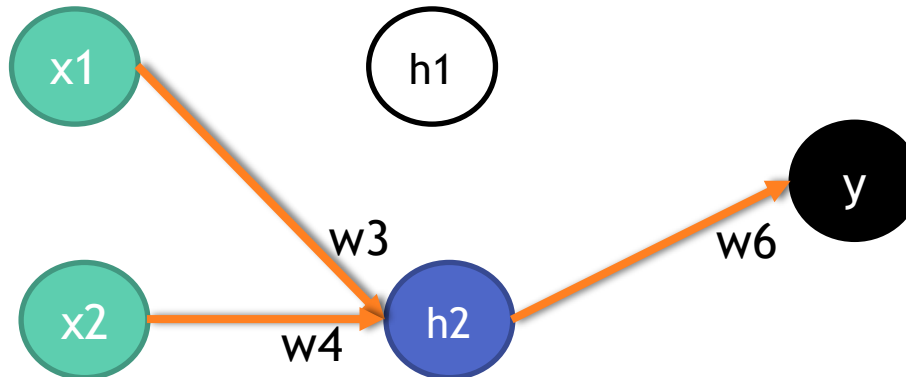
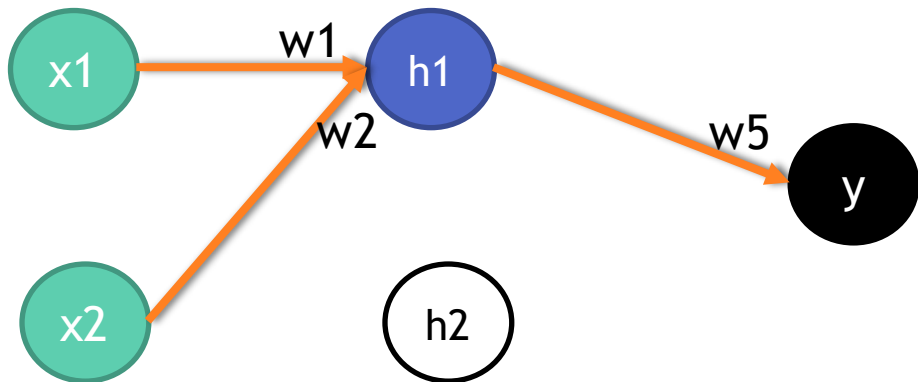


While testing



- While testing take all the weights from final set.
- We keep all nodes as if there is no drop out.
- The new formula for y is different now
- $y = \text{sigmoid}(w_5 \cdot h_1 + w_6 \cdot h_2)$

Let 0.5 be the probability of retaining a unit. We will see below structures while training



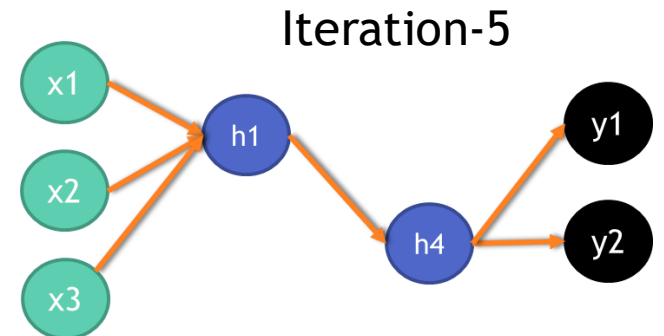
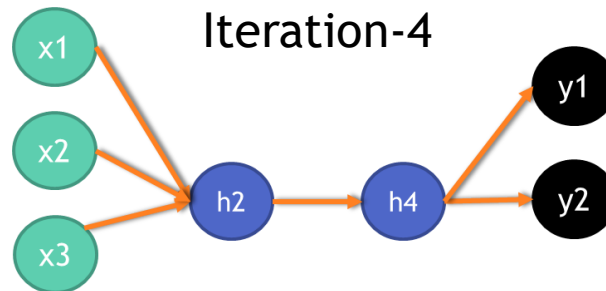
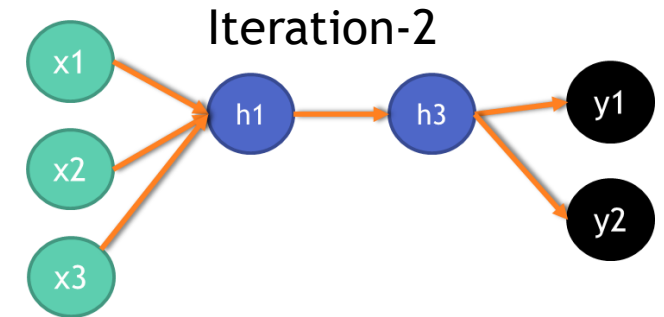
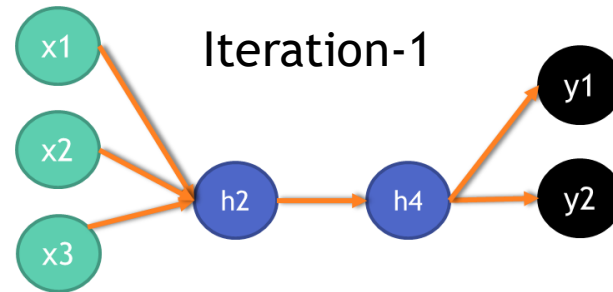
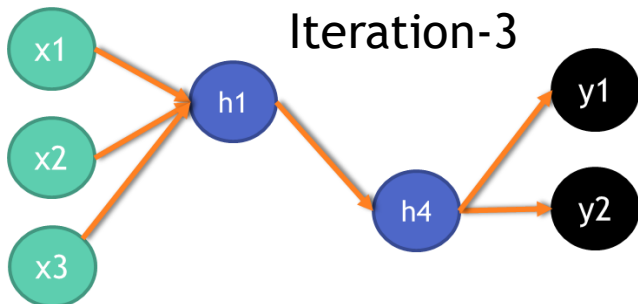
Final weights while testing

- While training
 - $y = \text{sigmoid}(w_5 * h_1)$ or
 - $y = \text{sigmoid}(w_6 * h_2)$
- While testing
 - $y = \text{sigmoid}(w_5 * h_1 + w_6 * h_2)$
- We may have to incorporate the effect of dropout on weights while testing
- h_1 and h_2 have retaining probability of “ p ”. They are not picked always. Their weights should not have full impact on y .
- Multiply weights by p while testing.
 - $y = \text{sigmoid}(p * w_5 * h_1 + p * w_6 * h_2)$
 - $y = \text{sigmoid}(0.5 * w_5 * h_1 + 0.5 * w_6 * h_2)$ (in our example $p=0.5$)

Weights sharing in Dropout

- Do not assume that the model is fixed and weights are trained for that network.
- The dropout happens at every iteration.

- Iteration 3 and iteration 5 have same network configuration
- After completing iteration-4, where do you get the weights for iteration-5?
- From iteration 3? This is nothing but concept of shared weights.



Why Dropout Works?

- Avoids Individual Dominance:
 - Dropout ensures that a single neuron is NOT dominating in the entire model. It may get dropped at any point. Other neurons will get a chance.
- Avoids Interdependencies:
 - If a hidden unit knows that all other hidden units are present, it can co-adapt to them for a given training data.
 - Since any neuron can be dropped at any iteration, every neuron has to act independently.
- No Redundant hidden units anymore
 - If a hidden unit has to work well alongside a randomly selected group of hidden units, it has to do something that is individually valuable.

LAB: Dropout Regularization

- Try Dropout regularization on MNIST data
- Build a model with several hidden nodes and layers. Apply dropout to avoid overfitting

Code: Dropout Regularization

```
from tensorflow.keras.layers import Dropout
model_rd = keras.Sequential()

model_rd.add(layers.Dense(256, activation='sigmoid', input_shape=(784,)))
model_rd.add(Dropout(0.7))

model_rd.add(layers.Dense(128, activation='sigmoid'))
model_rd.add(Dropout(0.6))

model_rd.add(layers.Dense(10, activation='softmax'))
model_rd.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
=====		
dense_6 (Dense)	(None, 256)	200960
dropout (Dropout)	(None, 256)	0
dense_7 (Dense)	(None, 128)	32896
dropout_1 (Dropout)	(None, 128)	0
dense_8 (Dense)	(None, 10)	1290
=====		

Total params: 235,146

Trainable params: 235,146

Non-trainable params: 0

Code: Dropout Regularization

```
model_rd.compile(loss='categorical_crossentropy', metrics=['accuracy'])
model_rd.fit(x_train, y_train, epochs=10)

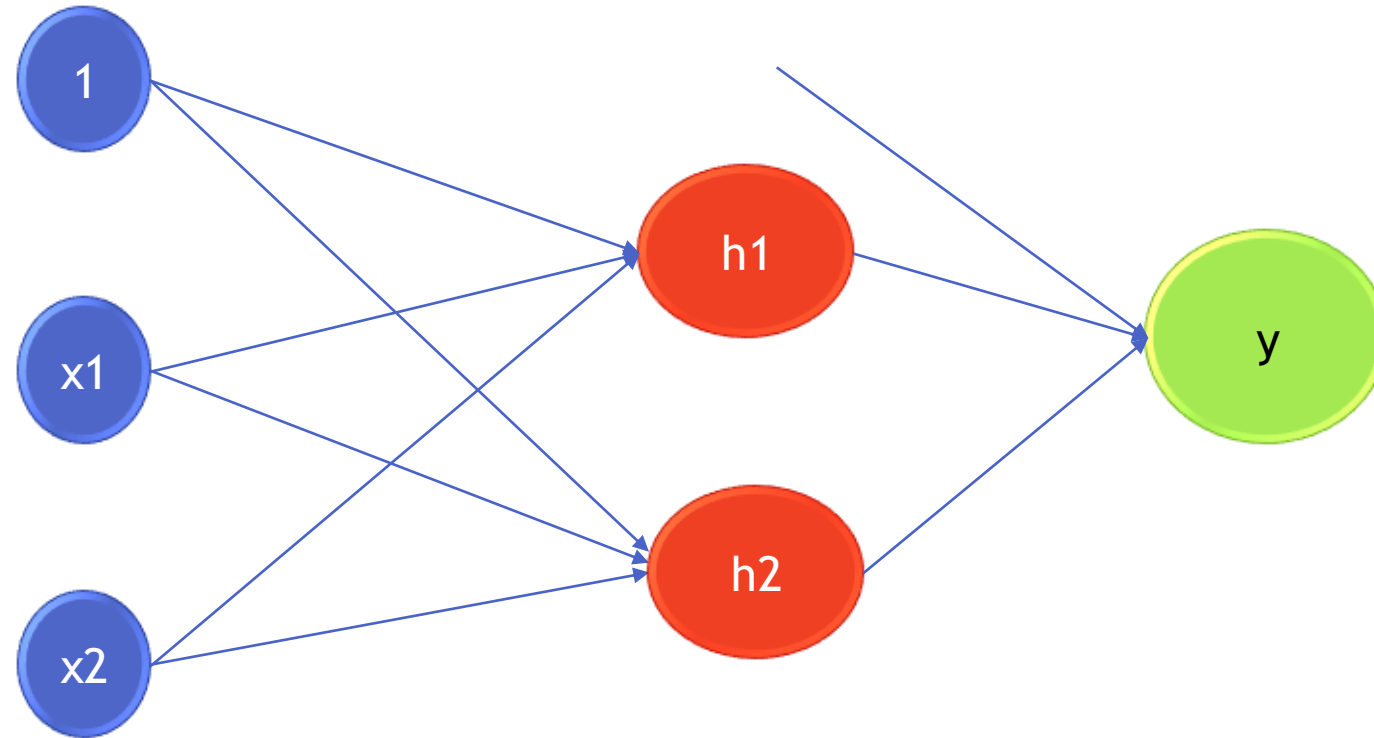
#Final Results
loss, acc = model_rd.evaluate(x_train, y_train, verbose=2)
print("Train Accuracy: {:.2f}%".format(100*acc))

loss, acc = model_rd.evaluate(x_test, y_test, verbose=2)
print("Test Accuracy: {:.2f}%".format(100*acc))
```

```
Epoch 1/10
1875/1875 [=====] - 8s 4ms/step - loss: 0.8384 - accuracy: 0.7293
Epoch 2/10
1875/1875 [=====] - 8s 4ms/step - loss: 0.4393 - accuracy: 0.8738
Epoch 3/10
1875/1875 [=====] - 8s 4ms/step - loss: 0.3759 - accuracy: 0.8959
Epoch 4/10
1875/1875 [=====] - 8s 4ms/step - loss: 0.3411 - accuracy: 0.9065
Epoch 5/10
1875/1875 [=====] - 8s 4ms/step - loss: 0.3213 - accuracy: 0.9141
```

Activation Functions

Activation Functions



$$y = g(\sum W_j g(\sum w_{jk} x_k))$$

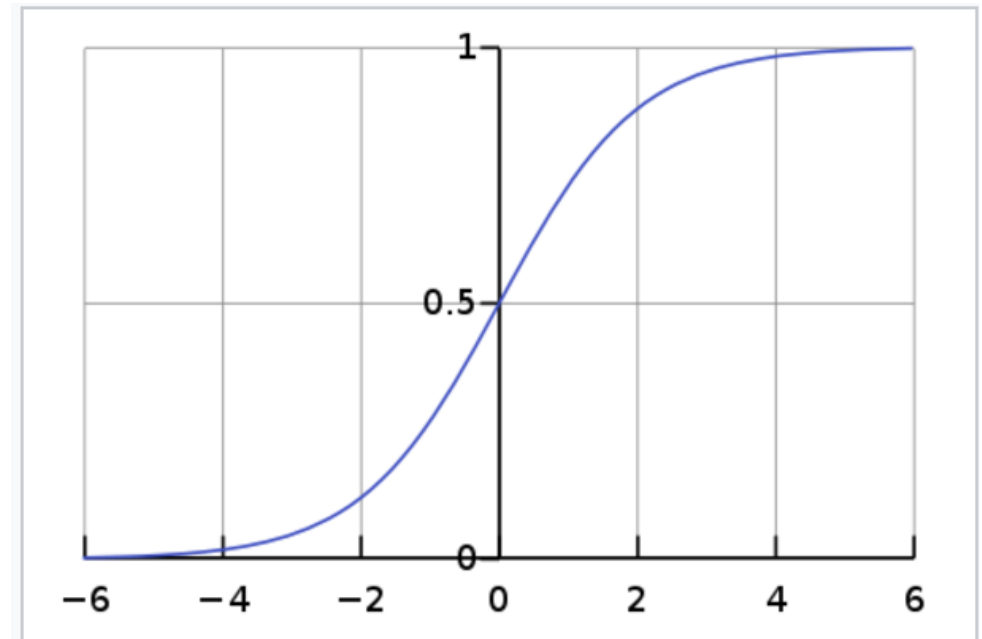
- In the above equation $g(x)$ is known as activation function

Activation Functions - Sigmoid

- Sigmoid function. Historically famous activation function
- Also known as Inverse logistic
- Works well for usual business related data
- Works well for a binary or multi class output
- it takes a real-valued number and “squashes” and outputs number between 0 and 1.
- Large negative numbers become 0 and large positive numbers become 1.

$$\text{sigmoid}(x) = \frac{e^x}{1 + e^x}$$

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$



Sigmoid- Drawbacks

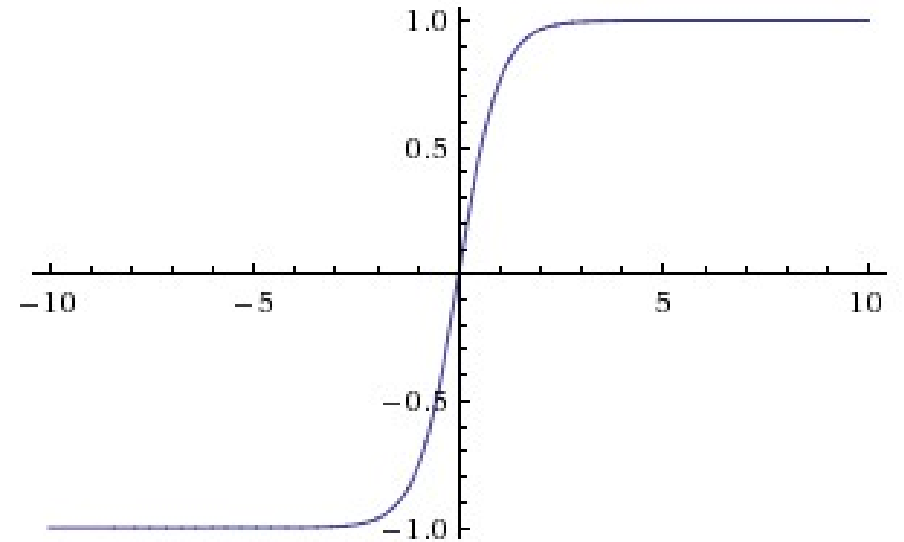
- Sigmoid output not zero centered.
 - The middle value is 0.5
 - Most of the times we normalise the data(zero centered) before building the neural network model.
 - It takes a lot of epochs(time) for convergence
- Is there a better activation function for faster convergence?

New Activation Function- TanH

- The TanH similar to sigmoid
- Squashes the real values between -1 and +1
- The output is zero centered.
- Preferred activation function for zero centered (normalized) data.

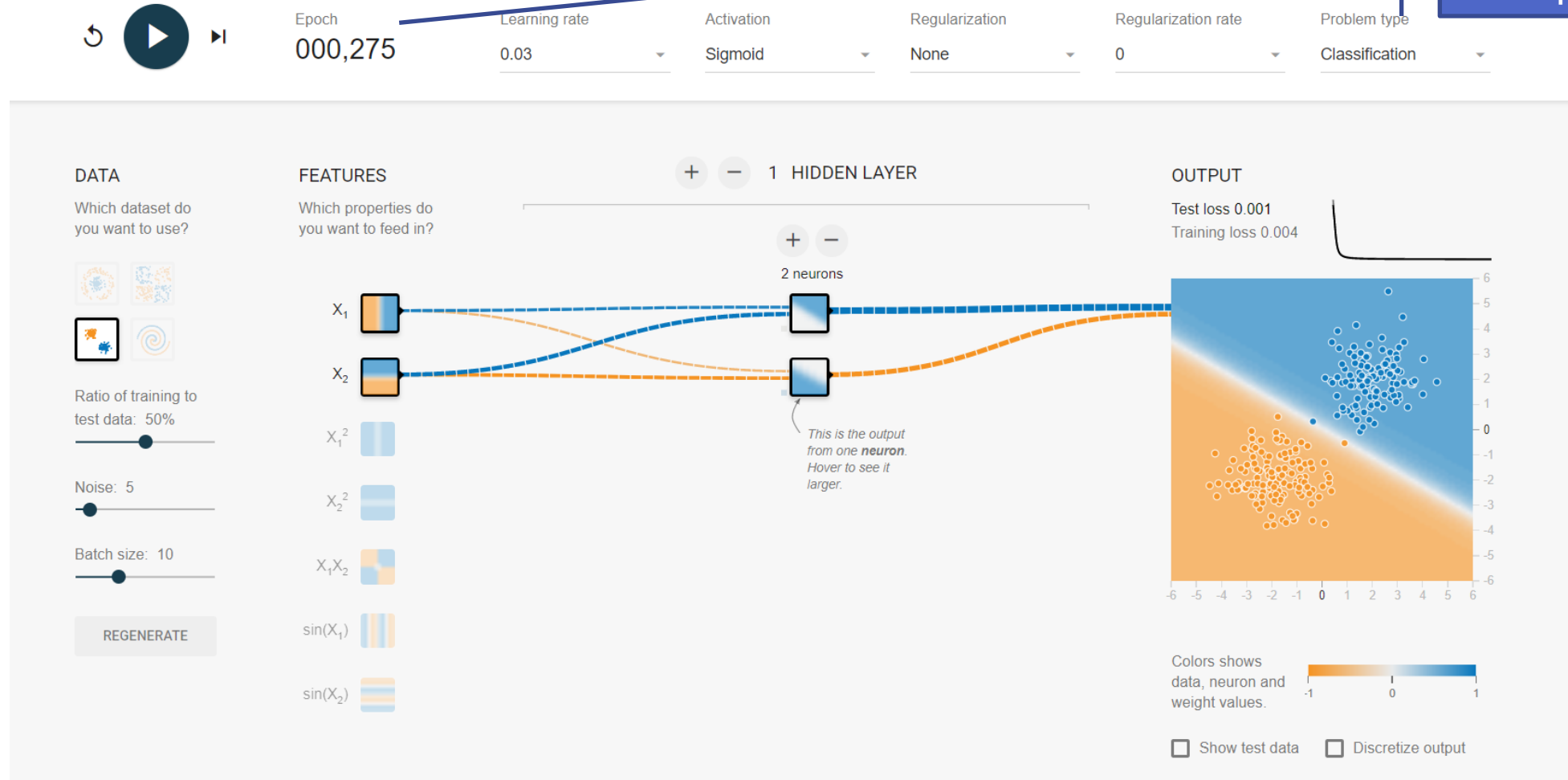
$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)}$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



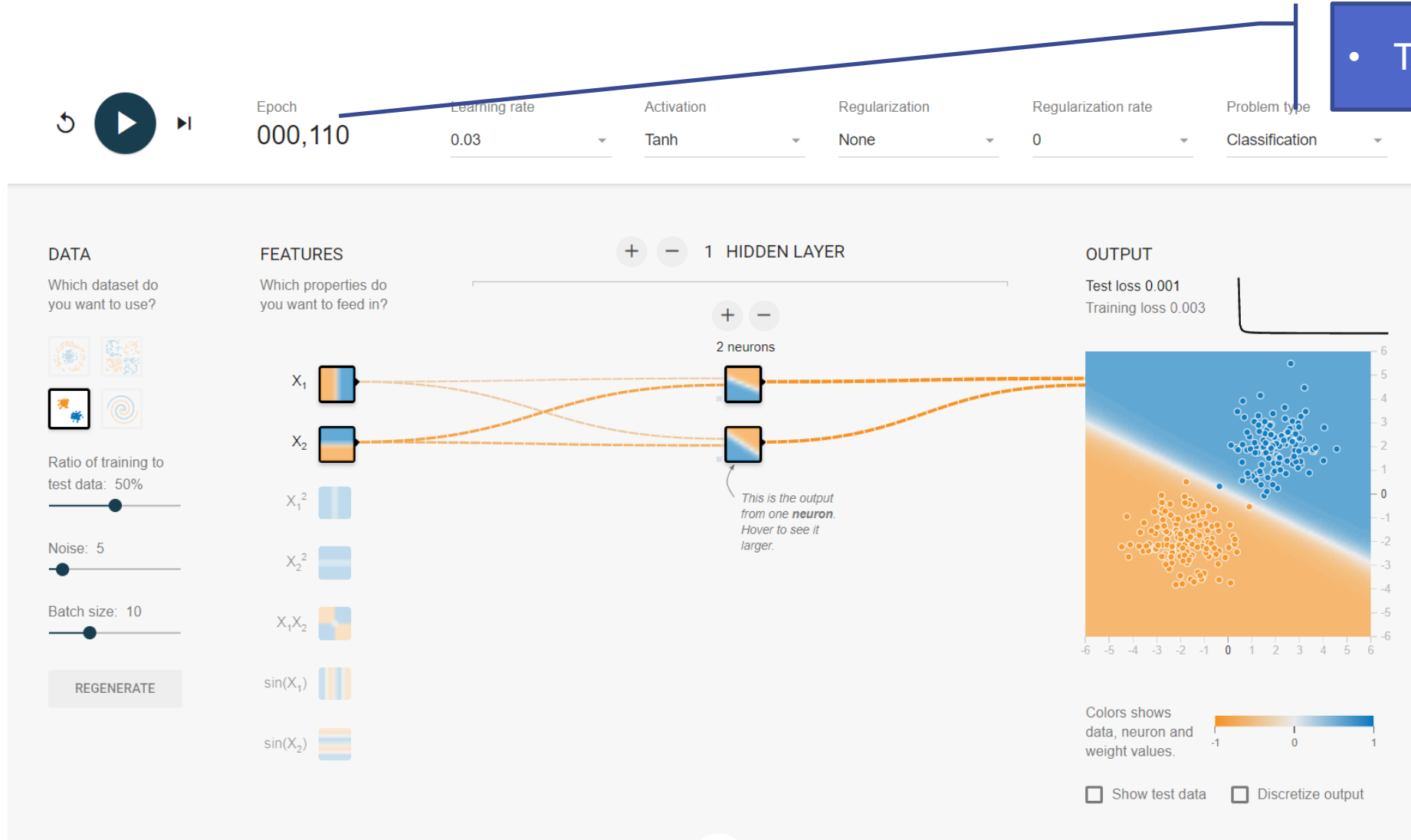
Demo: Sigmoid vs TanH

- Sigmoid takes 275 steps



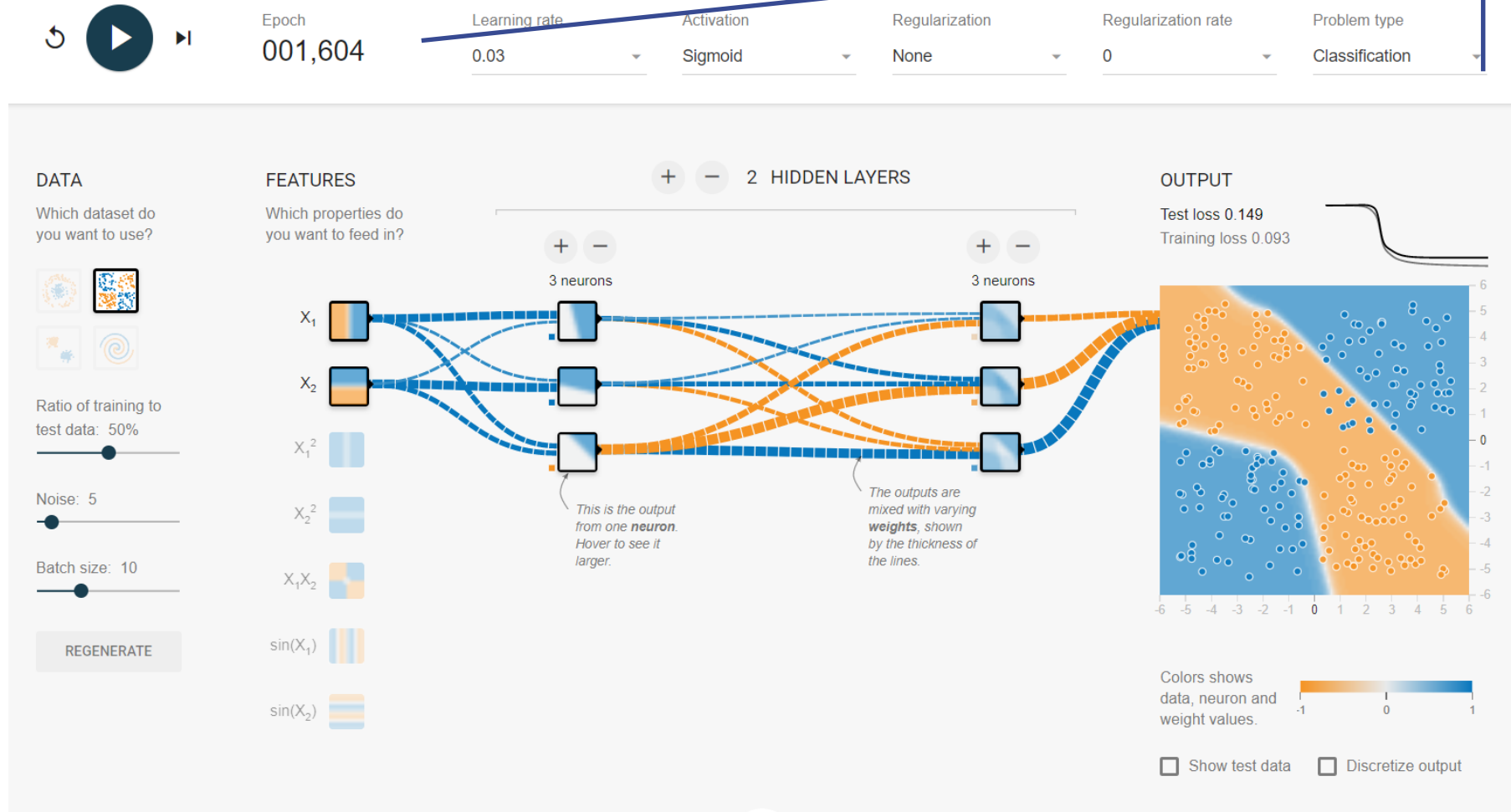
Demo: Sigmoid vs TanH

- Tanh takes less steps



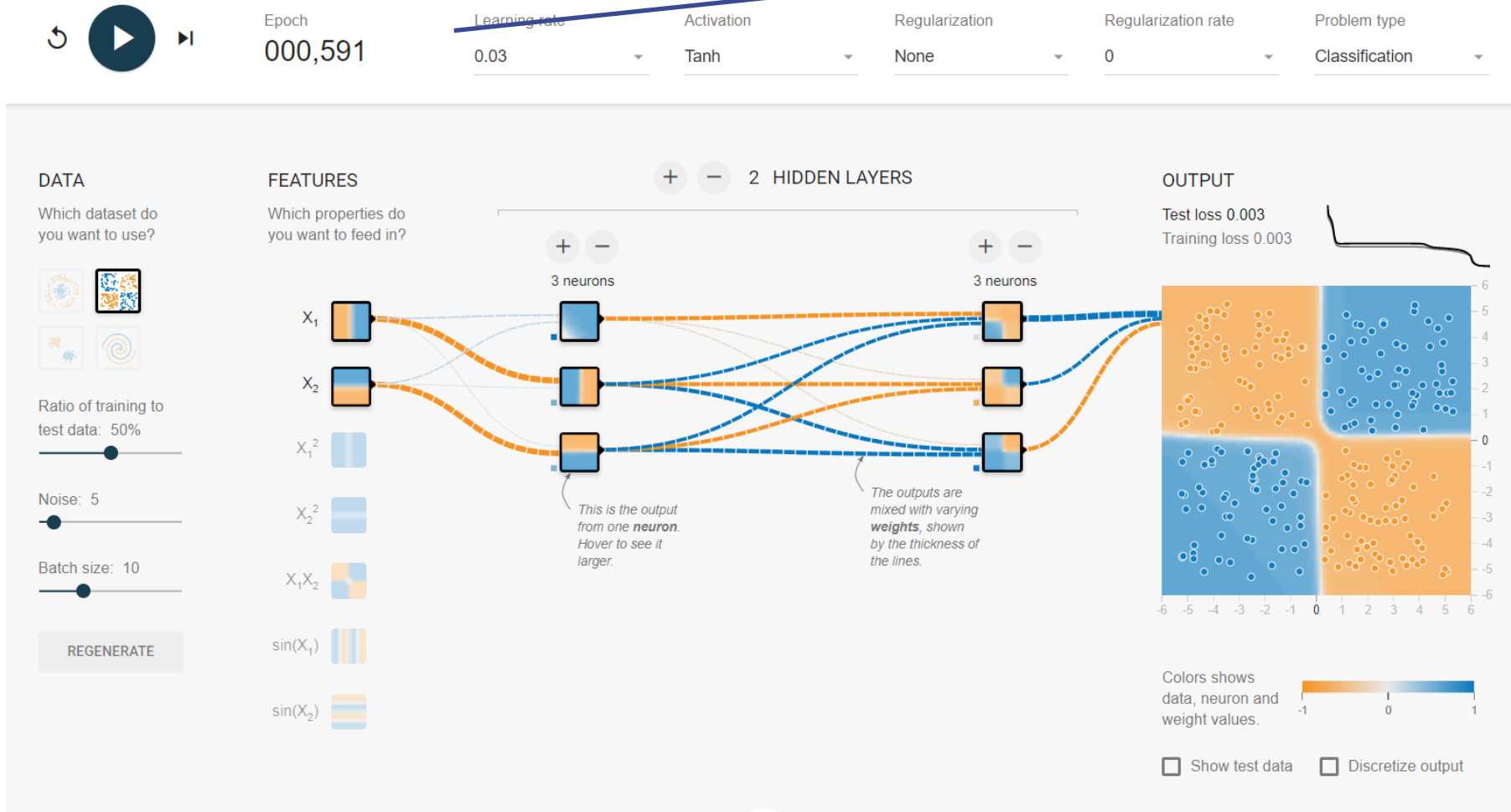
Demo: Sigmoid vs TanH

- Sigmoid takes 1,604 steps
- Error is still 15%



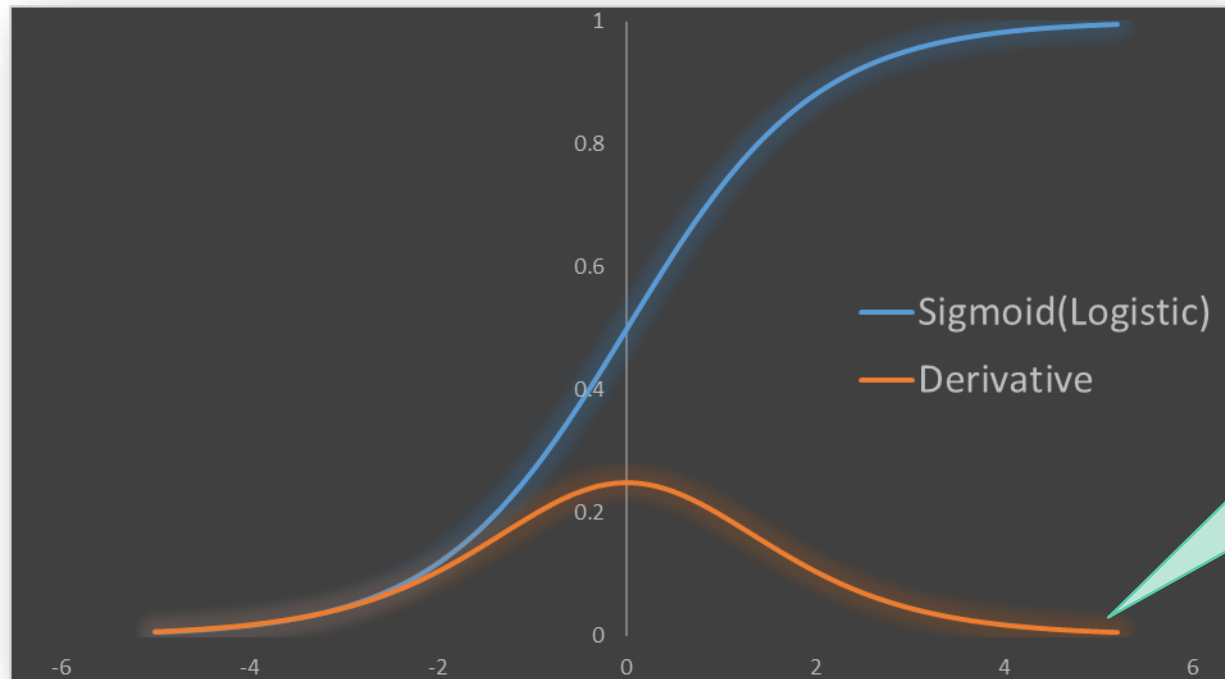
Demo: Sigmoid vs TanH

- TanH takes 600 steps
- Error is less than %



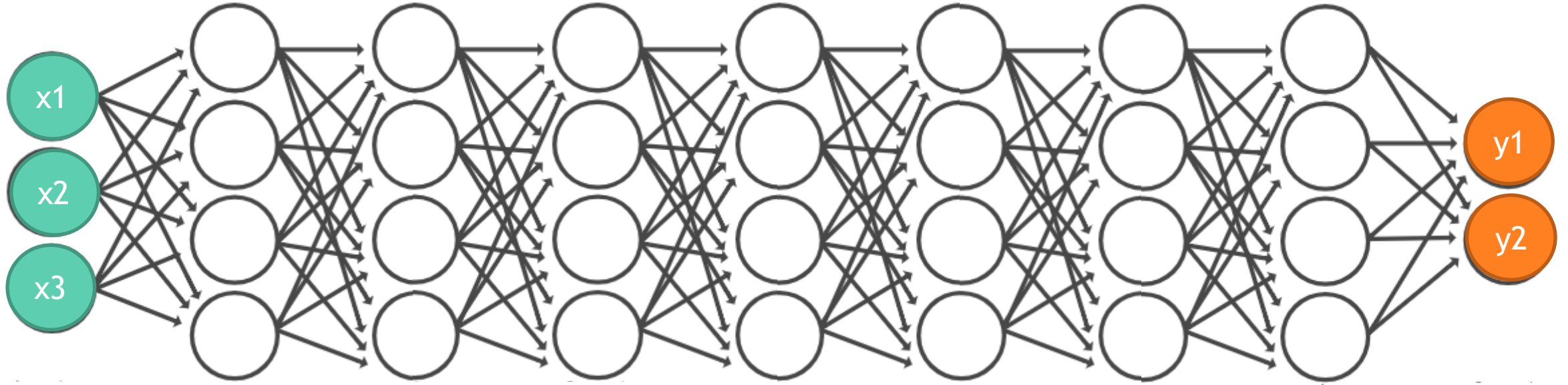
The problem of Vanishing gradients

- Your deep neural network might be suffering from a problem of vanishing gradients
- Computationally sigmoid values and gradient values and their multiplications are very small
- For a deep network with many hidden layers sigmoid gradients vanish very quickly



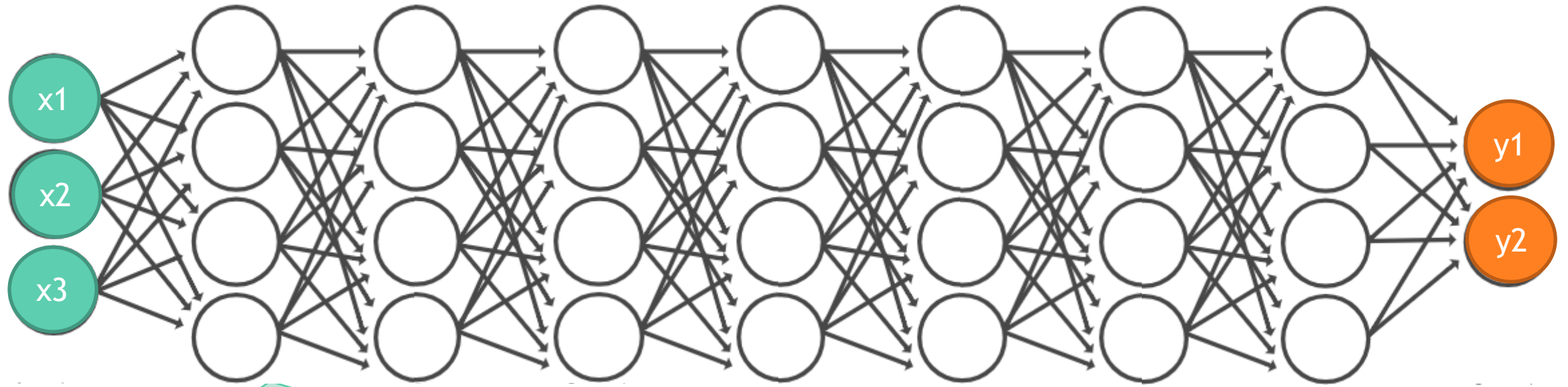
The derivative of sigmoid is zero on both ends

The problem of vanishing gradients



The gradient values are already low here

The problem of vanishing gradients

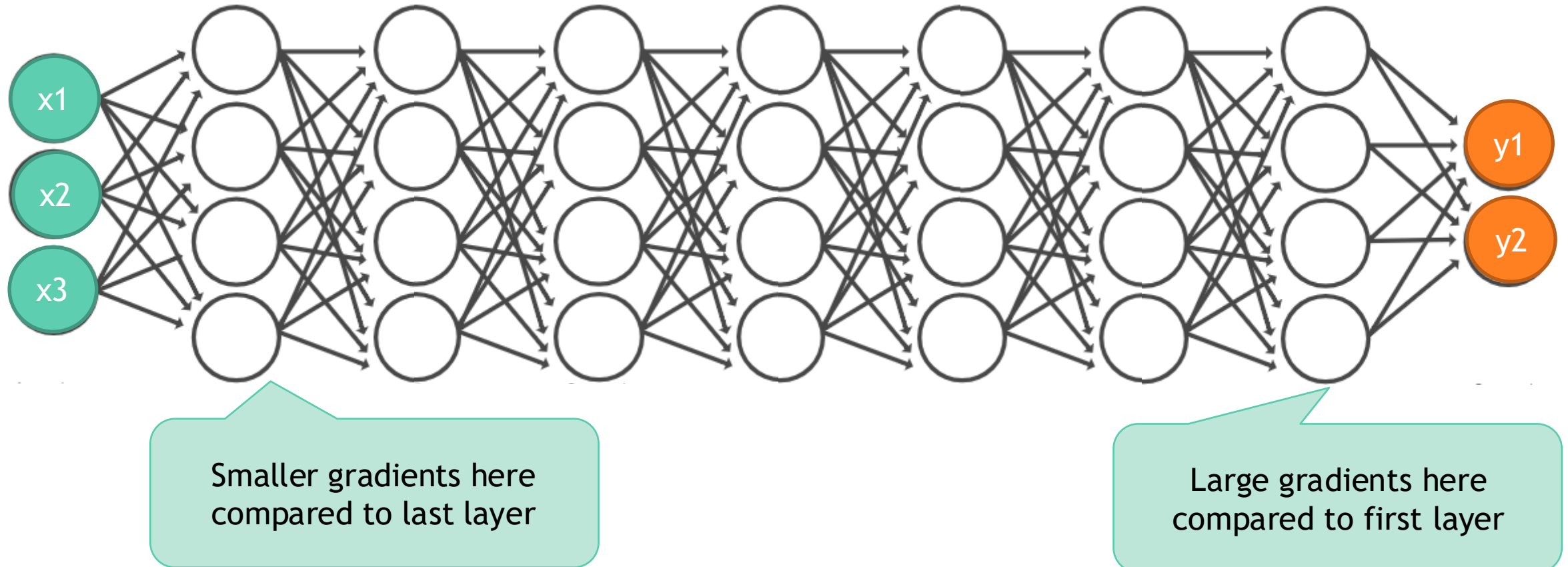


If the values of gradients at final layers are already low, then gradient will be almost zero for the starting layers

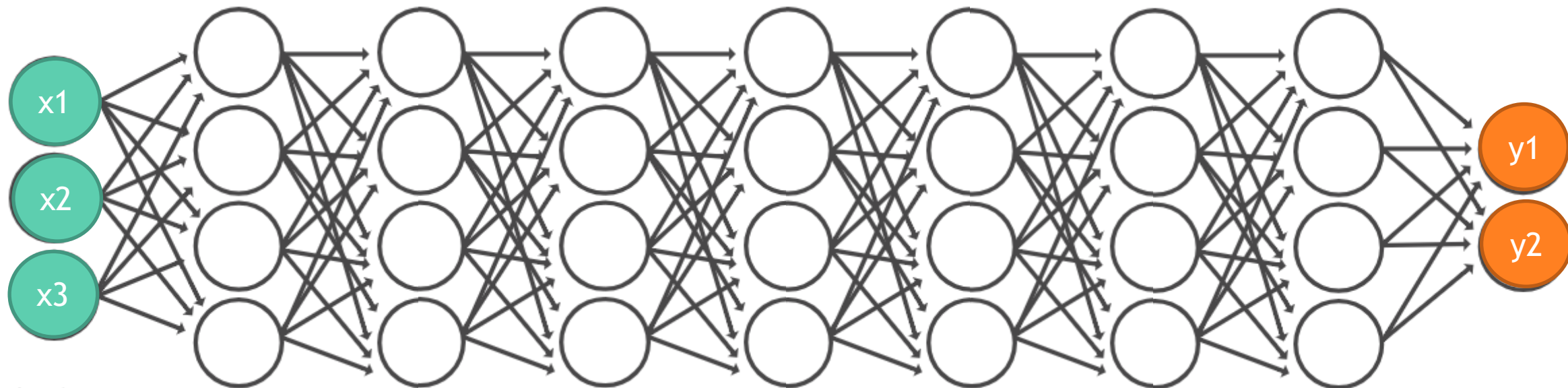
The gradient values are already low here

- With small values in the matrix and several matrix multiplications (from layer k to 1) the gradient values shrink exponentially fast. Eventually vanishing completely after a few layers.

Other Observation - Vanishing gradients



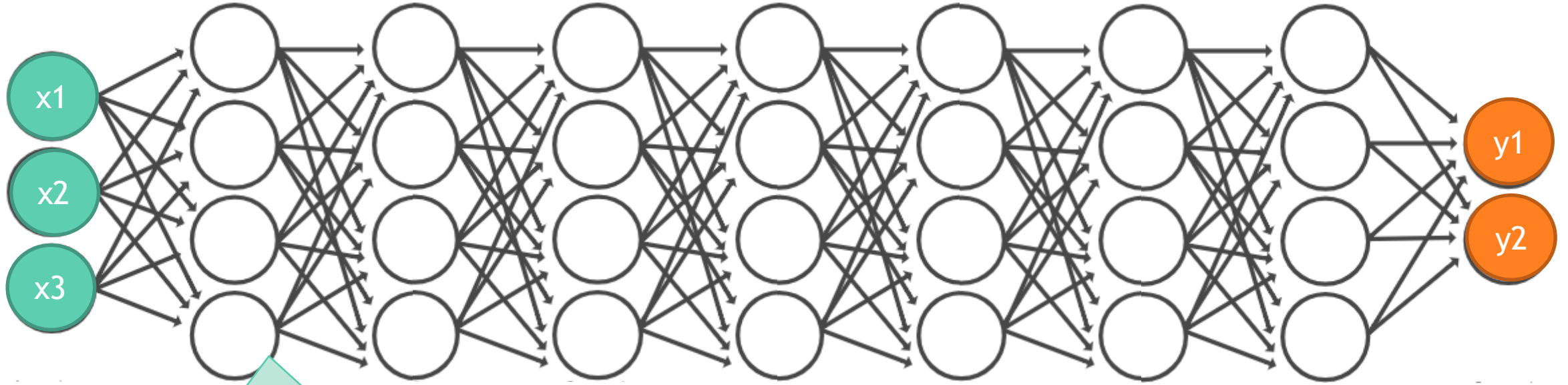
Other Observation - Vanishing gradients



- Smaller gradients here compared to last layer
- Learning is slower

- Large gradients here compared to first layer
- Learning is faster here

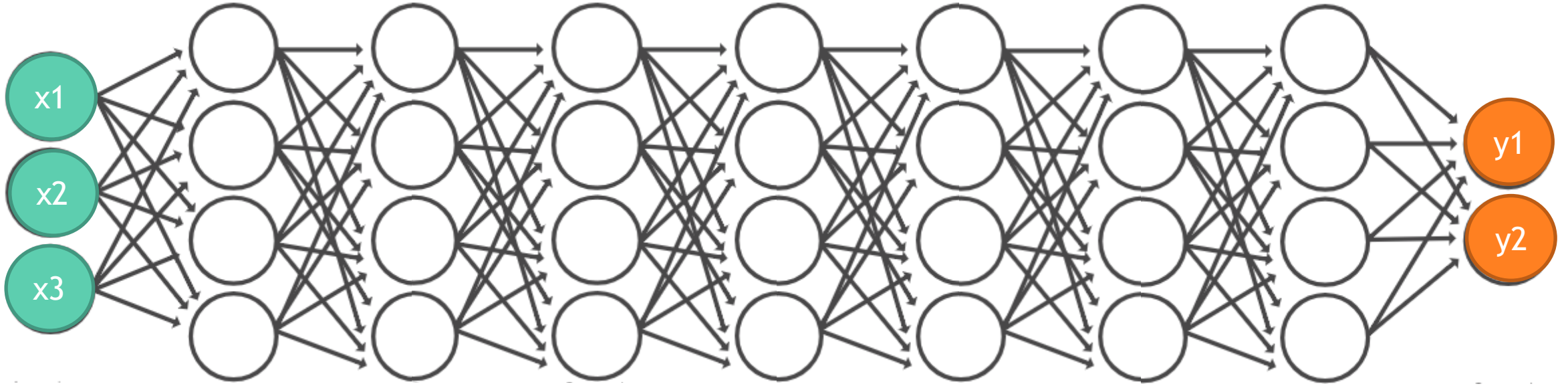
Other Observation - Vanishing gradients



- Smaller gradients here compared to last layer
- Learning is slower
- No movement in the weights

- Large gradients here compared to first layer
- Learning is faster here
- Converged quickly

Other Observation - Vanishing gradients

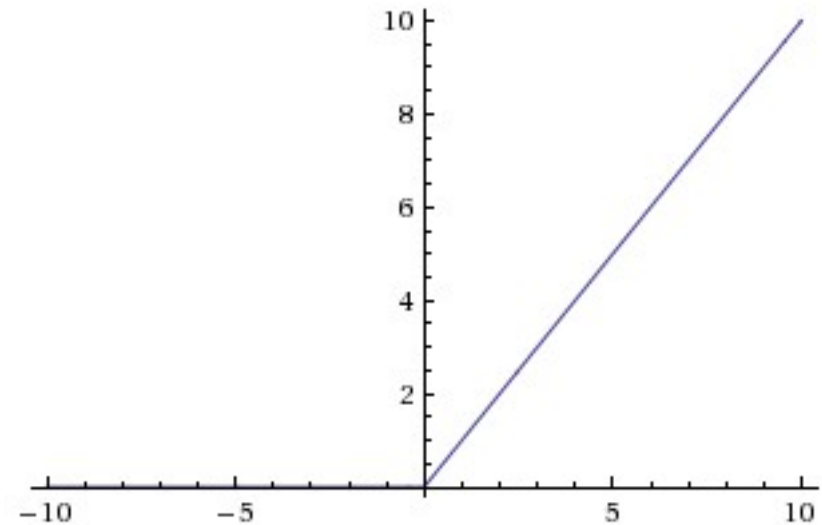


- Not just sigmoid, tanh also suffers from vanishing gradients problem
- We need a new activation function that has larger gradients
- We need an activation function that can penetrate from final layer till the first layer
- Is there an activation function to handle vanishing gradients problem.

ReLU - The Rectified Linear Unit

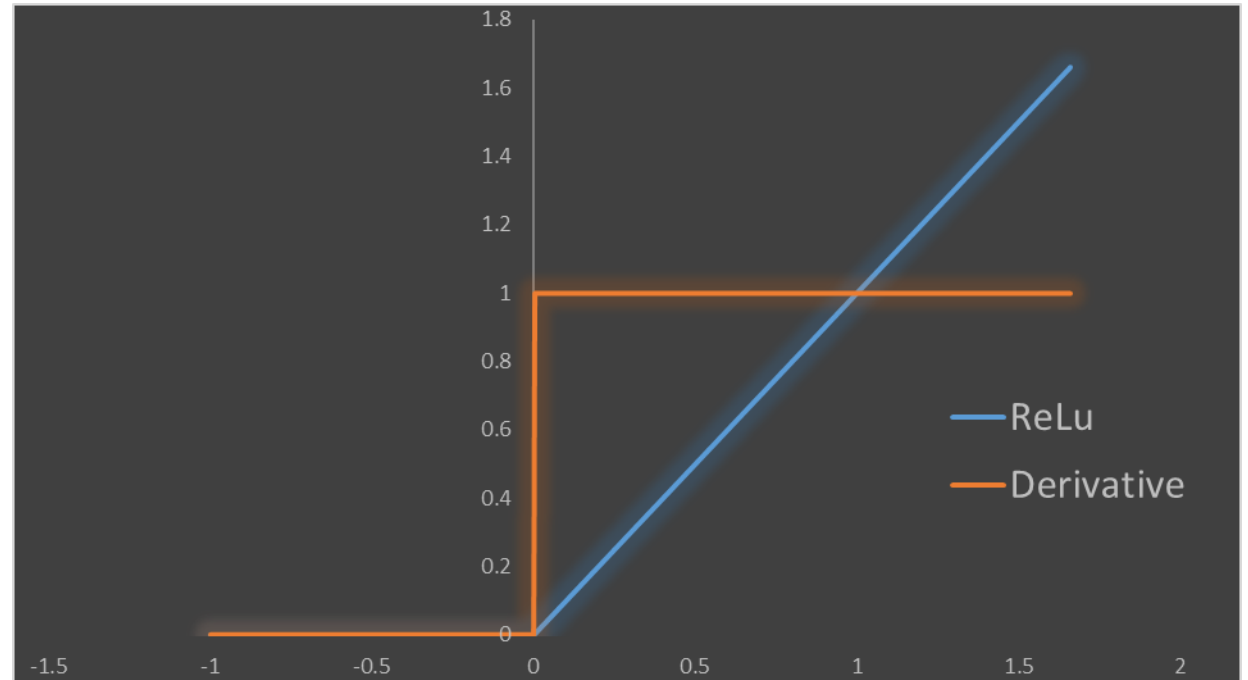
- Very popular activation function in recent times
- $f(x) = \max(0, x)$
- In other words, the activation is simply thresholded at zero
- Works very well for a certain class of problems
- Doesn't have vanishing gradient problem. It can be used for modelling real values

$$f(x) = \max(0, x)$$
$$f(x) = \log(1 + e^x)$$

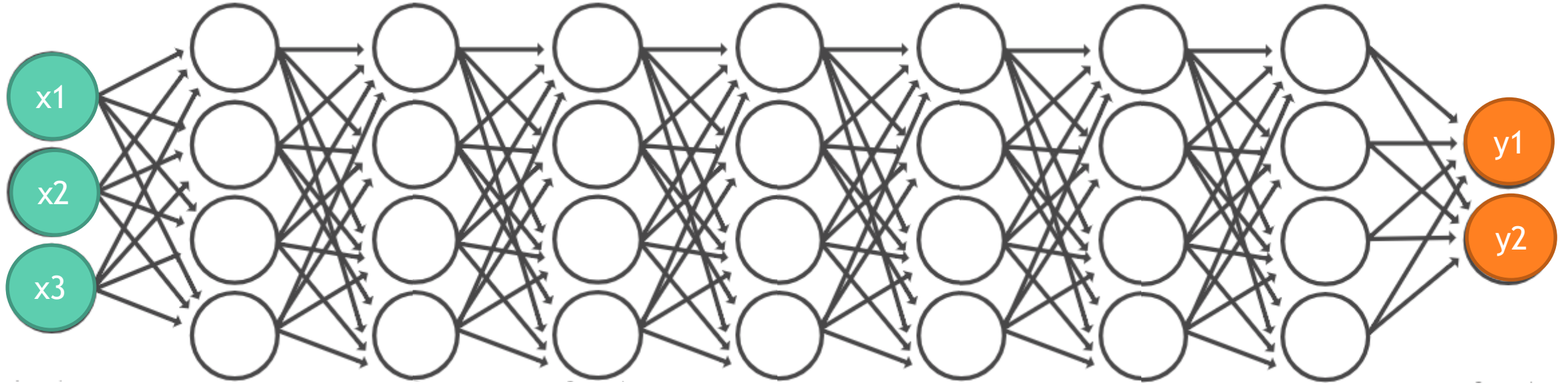


ReLU – Handles Vanishing gradients

- The derivative of ReLU is 1 if $x > 0$ else zero
- The derivatives are not small fractions
- Matrix multiplications won't result in vanishing gradients



ReLU resolves vanishing gradients issue



- The gradients won't shrink from final layer to previous layers. As we are **NOT** multiplying them with fractions.
- ReLU is used in deep neural networks with too many hidden layers

Activation function has last layer

- In classification problems ReLU may not be suitable for output layer
- ReLU gives the output as 0 or positive continuous number. ReLU output can be 0 or 0.3 or 0.5 or 1.8 or 2 or even 20. In classification, output values are either 1 or 0.
- We may need to go back to sigmoid activation for binary output
- For multiclass problems the output is 1 for the current class and 0 for rest all classes.
- Softmax is used for multiclass output. It is just like sigmoid for multiclass

$$y1 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$$

$$y2 = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$$

$$y3 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$$

$$yk = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$$

SoftMax for the output layer

- SoftMax is just a generalization of sigmoid function for multiclass output
- Softmax= Output of class j divided by overall sum of all classes.

$$\text{softmax}(x)_j = \frac{e^{x_j}}{\sum_{k=1}^K e^{x_k}}$$

Lets take an example output with 4 classes

$$\text{Output Class 1} = \frac{e^{w_1x}}{e^{w_1x} + e^{w_2x} + e^{w_3x} + e^{w_4x}}$$

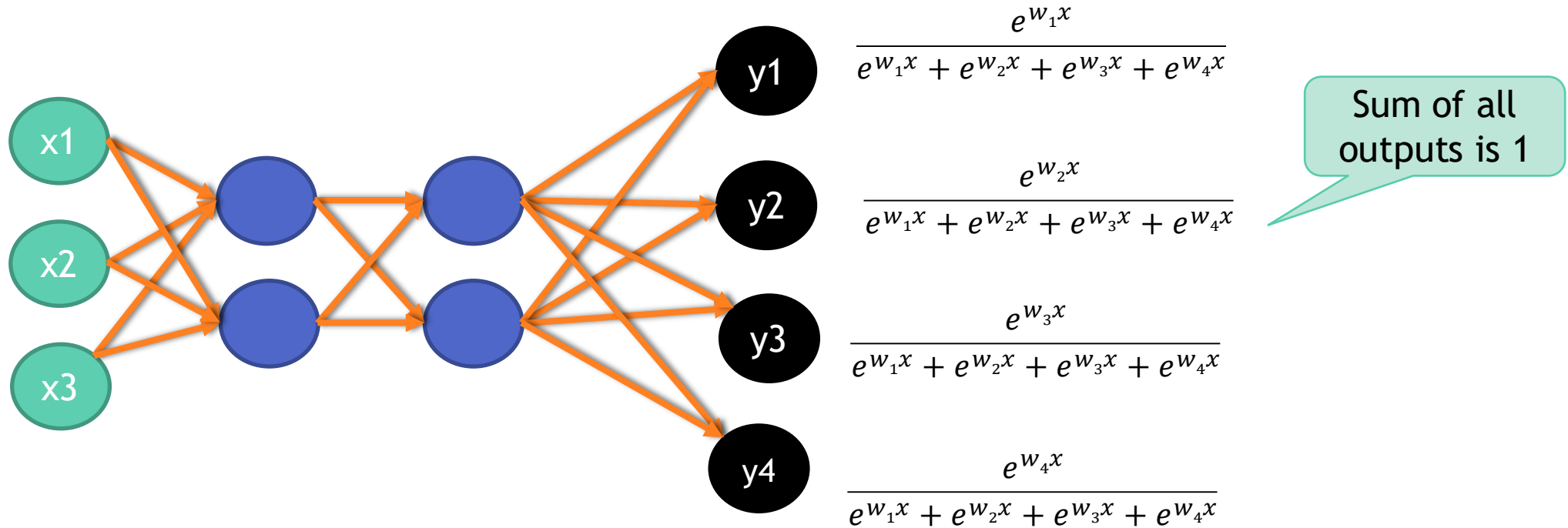
$$\text{Output Class 2} = \frac{e^{w_2x}}{e^{w_1x} + e^{w_2x} + e^{w_3x} + e^{w_4x}}$$

$$\text{Output Class 3} = \frac{e^{w_3x}}{e^{w_1x} + e^{w_2x} + e^{w_3x} + e^{w_4x}}$$

$$\text{Output Class 4} = \frac{e^{w_4x}}{e^{w_1x} + e^{w_2x} + e^{w_3x} + e^{w_4x}}$$

SoftMax for the output layer

- You can see SoftMax as summarization of “one vs all” models using sigmoid and scaling it to one.



LAB: Activation functions

- Try different activation functions on MNIST example

```

model2 = keras.Sequential()
model2.add(layers.Dense(15, activation='sigmoid', input_shape=(784,)))
model2.add(layers.Dense(15, activation='relu'))
model2.add(layers.Dense(15, activation='tanh'))
model2.add(layers.Dense(15, activation='relu'))
model2.add(layers.Dense(10, activation='softmax'))
model2.summary()

```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
=====		
dense_9 (Dense)	(None, 15)	11775
dense_10 (Dense)	(None, 15)	240
dense_11 (Dense)	(None, 15)	240
dense_12 (Dense)	(None, 15)	240
dense_13 (Dense)	(None, 10)	160
=====		
Total params: 12,655		
Trainable params: 12,655		
Non-trainable params: 0		

Code: Activation Functions

Learning rate

Learning rate

- The weight updating in neural networks

$$W_{jk} := W_{jk} + \Delta W_{jk}$$

$$\text{where } \Delta W_{jk} = \eta \cdot y_j \delta_k$$

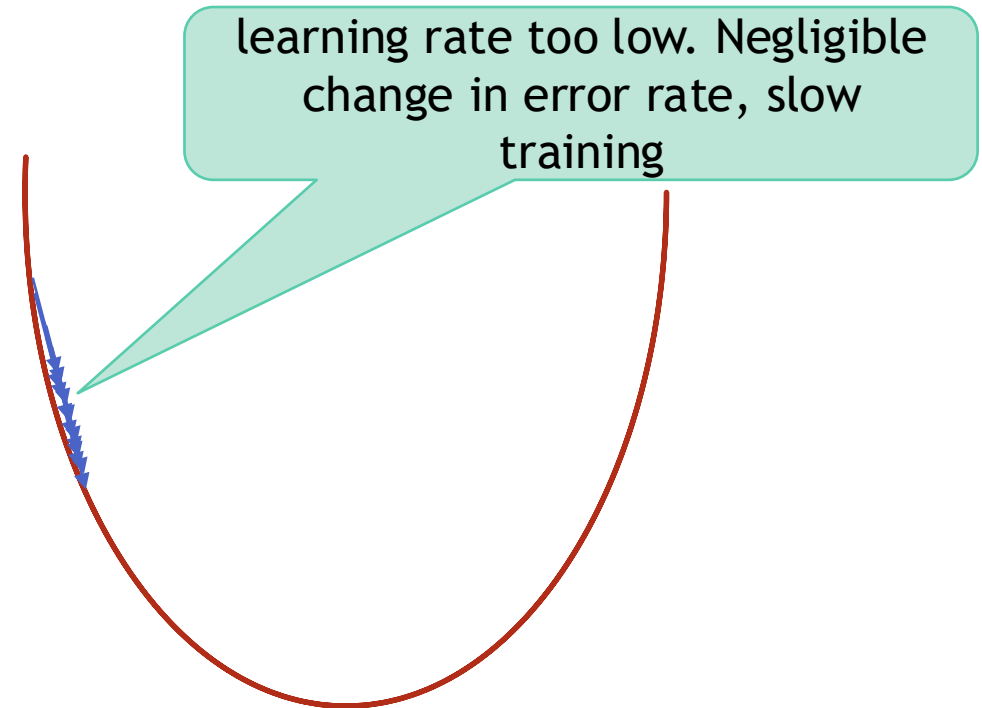
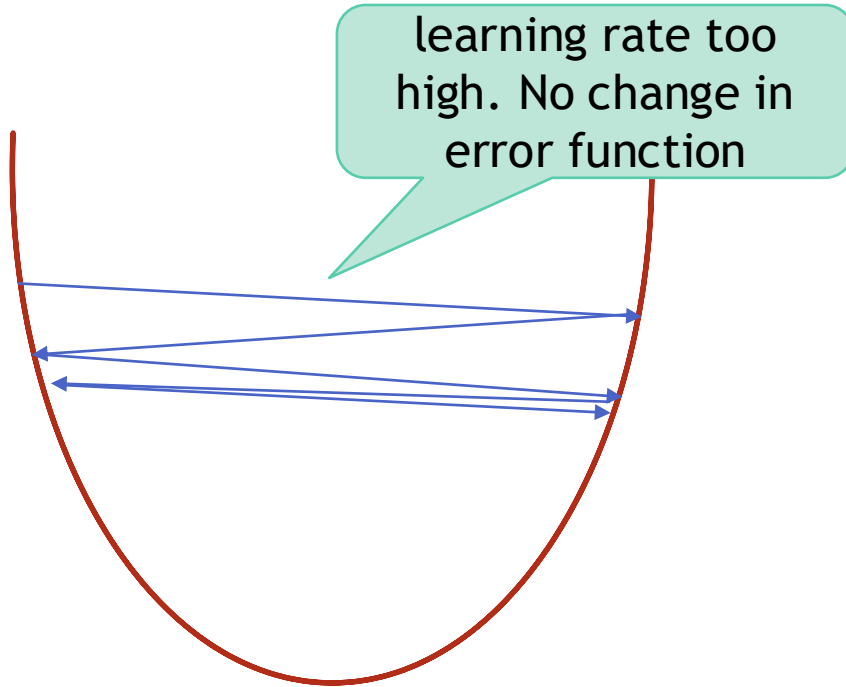
where η is the learning parameter

$$W_{jk} := W_{jk} + \eta \cdot y_j \delta_k$$

- Instead of just updating the weights based on actual calculations, we will manually setup weight update parameter η
- This will make the weights move by a factor η . How fast /slow you want to move the weights
- Read it as Step-size for weights
- The speed at which the Neural Network arrives to minimum.

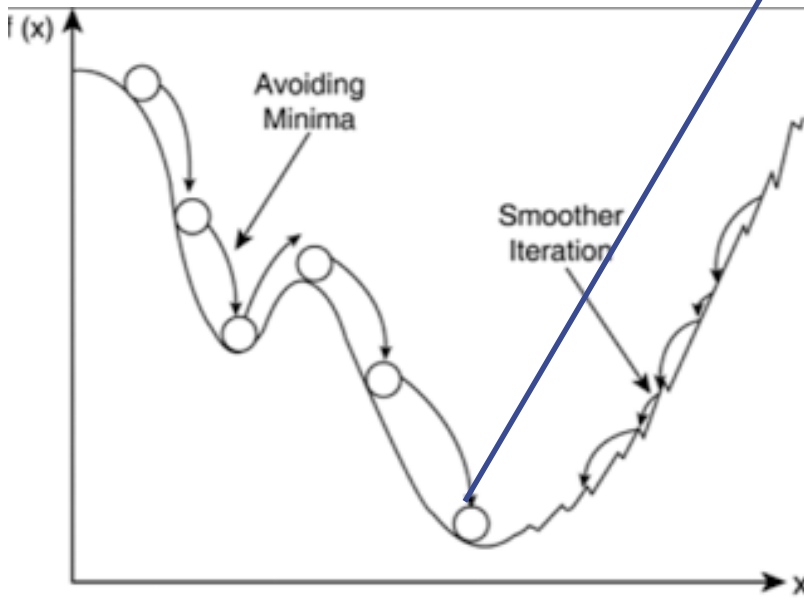
Learning Rate tuning

- If the learning rate is too high, the system will either oscillate about the true solution or it will diverge completely.
- If the step-size is too low, the system will take a long time to converge on the final solution.
- Generally start with a large value, reduce it gradually.



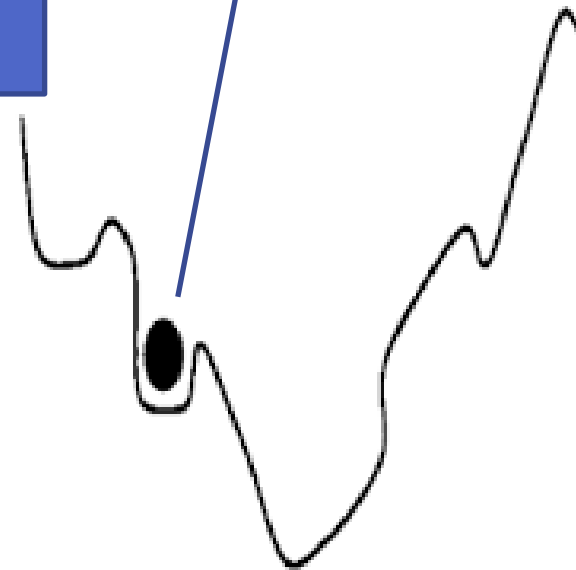
Learning Rate is Important

- For the actual practical problems the error surfaces are very complex
- Learning rate is very important to avoid local minima



- Optimal learning rate
- Avoids local minima
- High chance of ending at global minima

- Small learning rate
- Small steps towards minima
- Has a risk of ending up in local minima



Demo: Learning rate



Epoch
001,032

Learning rate
10

Activation
Tanh

Regularization
None

Regularization rate
0

Problem type
Classification

- High learning rate
- High error

DATA

Which dataset do you want to use?



Ratio of training to test data: 60%

Noise: 20

Batch size: 10

REGENERATE

FEATURES

Which properties do you want to feed in?

X_1
 X_2
 X_1^2
 X_2^2
 X_1X_2
 $\sin(X_1)$
 $\sin(X_2)$

+ - 2 HIDDEN LAYERS

+ -

4 neurons

+ -

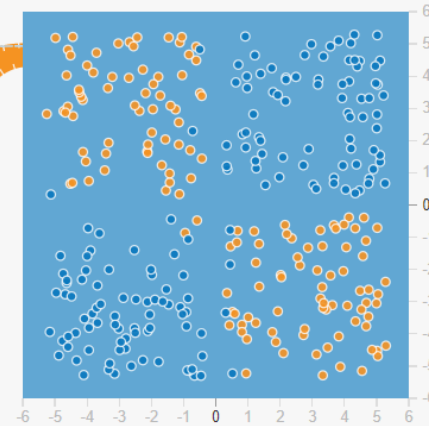
4 neurons

This is the output from one **neuron**. Hover to see it larger.

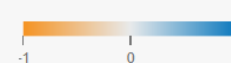
The outputs are mixed with varying **weights**, shown by the thickness of the lines.

OUTPUT

Test loss 0.940
Training loss 0.940



Colors shows data, neuron and weight values.

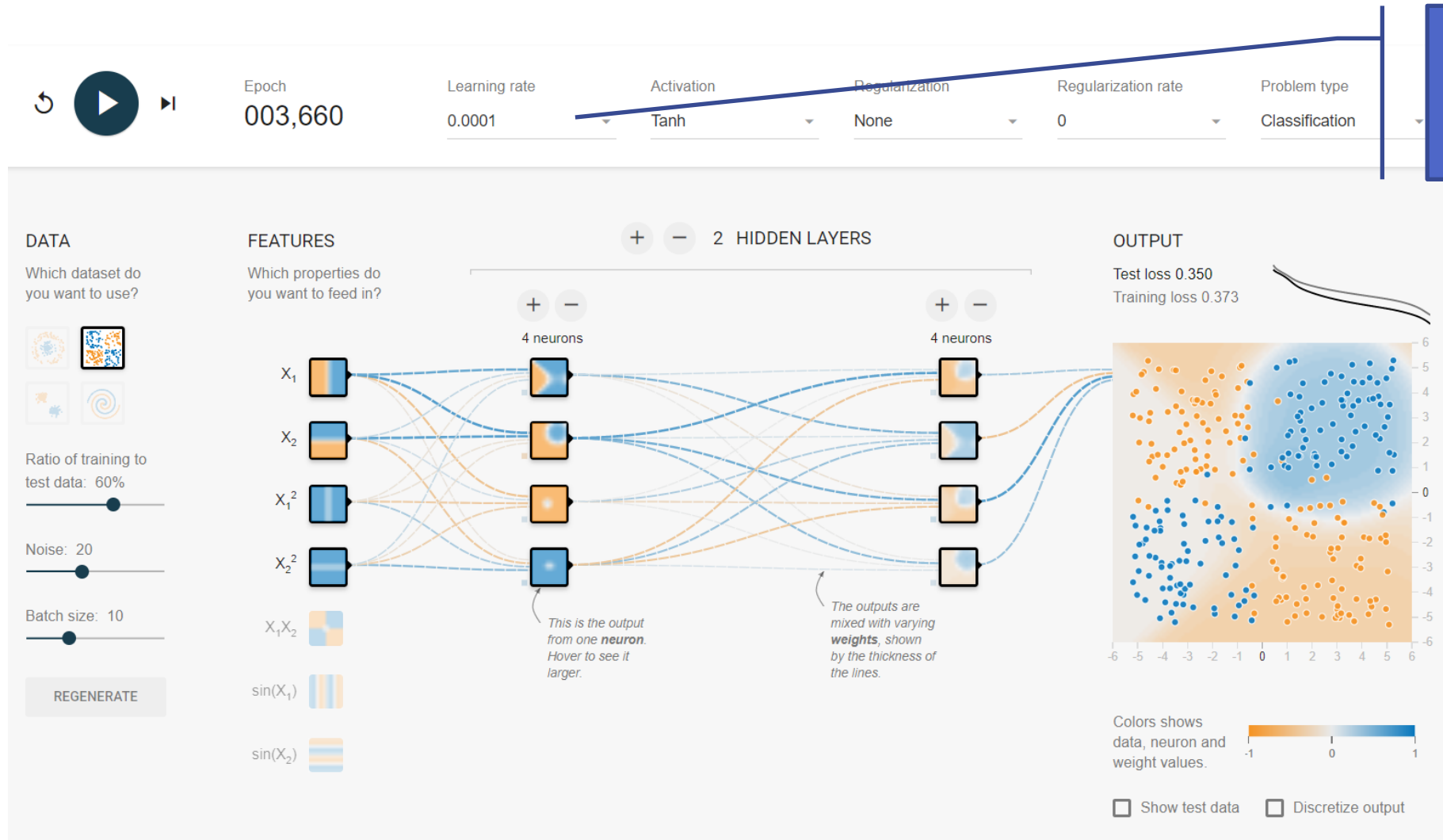


☐ Show test data

☐ Discretize output

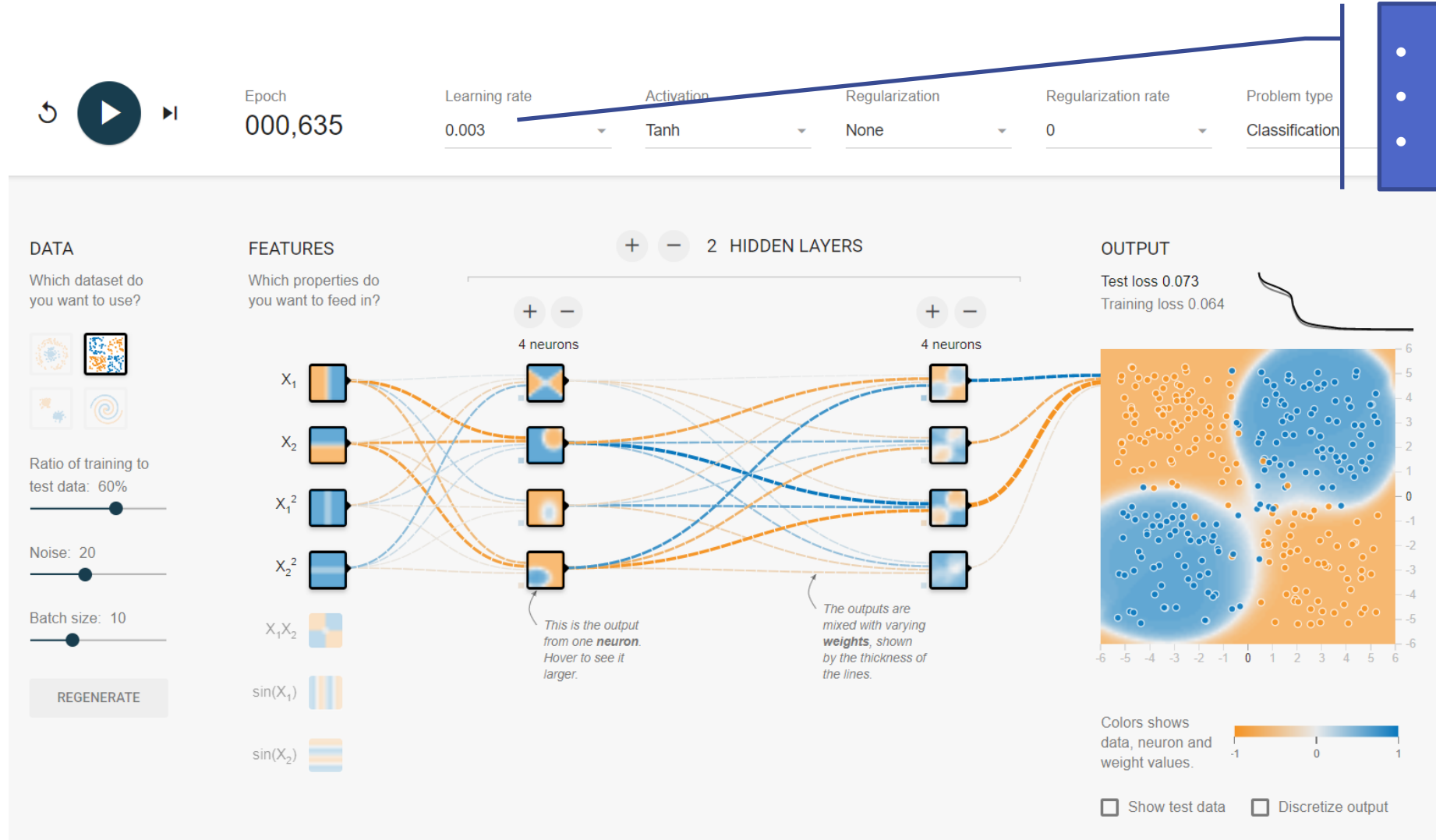
- No Improvement in the error

Demo: Learning rate



- Very low learning rate
- Too many steps
- Takes a lot of time to converge

Demo: Learning rate



- Optimal learning rate
- Low error
- Less steps

LAB: Learning Rate

- Build a deep NN model with high learning rate. Observe the impact on the target
- Build a model with a very low learning rate. Observe the impact
- Finetune learning rate and build an optimal model.

Code: Learning Rate

```
#High Learning Rate
```

```
opt_new = tf.keras.optimizers.SGD(learning_rate=10)
```

```
model3.compile(optimizer=opt_new, loss='categorical_crossentropy', metrics=['accuracy'])
```

```
model3.fit(x_train, y_train, epochs=20)
```

Epoch 1/20

1875/1875 [=====] - 2s 1ms/step - loss: 2.4789 - accuracy: 0.1011

Epoch 2/20

1875/1875 [=====] - 2s 1ms/step - loss: 2.4709 - accuracy: 0.1001

Epoch 3/20

1875/1875 [=====] - 2s 1ms/step - loss: 2.4705 - accuracy: 0.1012

Epoch 4/20

1875/1875 [=====] - 2s 1ms/step - loss: 2.4730 - accuracy: 0.1027

Epoch 5/20

1875/1875 [=====] - 2s 1ms/step - loss: 2.3365 - accuracy: 0.1402

Epoch 6/20

1875/1875 [=====] - 2s 1ms/step - loss: 2.1564 - accuracy: 0.1883

Epoch 7/20

1875/1875 [=====] - 2s 1ms/step - loss: 2.1322 - accuracy: 0.1921

Code: Learning Rate

```
#Low learning rate
opt_new = tf.keras.optimizers.SGD(learning_rate=0.00001)
model3.compile(optimizer=opt_new, loss='categorical_crossentropy', metrics=['accuracy'])
model3.fit(x_train, y_train, epochs=20)
```

Epoch 1/20

1875/1875 [=====] - 2s 1ms/step - loss: 2.4366 - accuracy: 0.1022

Epoch 2/20

1875/1875 [=====] - 2s 1ms/step - loss: 2.4329 - accuracy: 0.1022

Epoch 3/20

1875/1875 [=====] - 2s 1ms/step - loss: 2.4293 - accuracy: 0.1022

Epoch 4/20

1875/1875 [=====] - 2s 1ms/step - loss: 2.4259 - accuracy: 0.1022

Epoch 5/20

1875/1875 [=====] - 2s 1ms/step - loss: 2.4225 - accuracy: 0.1022

Epoch 6/20

1875/1875 [=====] - 2s 1ms/step - loss: 2.4192 - accuracy: 0.1022

Epoch 7/20

1875/1875 [=====] - 2s 1ms/step - loss: 2.4161 - accuracy: 0.1022

Epoch 8/20

1875/1875 [=====] - 2s 1ms/step - loss: 2.4130 - accuracy: 0.1022

Epoch 9/20

1875/1875 [=====] - 2s 1ms/step - loss: 2.4101 - accuracy: 0.1022

Code: Learning Rate

```
#Optimal learning rate
opt_new = tf.keras.optimizers.SGD(learning_rate=0.01)
model3.compile(optimizer=opt_new, loss='categorical_crossentropy', metrics=['accuracy'])
model3.fit(x_train, y_train, epochs=20)
```

```
Epoch 1/20
1875/1875 [=====] - 2s 1ms/step - loss: 2.2634 - accuracy: 0.2964
Epoch 2/20
1875/1875 [=====] - 2s 1ms/step - loss: 2.0585 - accuracy: 0.5070
Epoch 3/20
1875/1875 [=====] - 2s 1ms/step - loss: 1.6615 - accuracy: 0.6002
Epoch 4/20
1875/1875 [=====] - 2s 1ms/step - loss: 1.2712 - accuracy: 0.6823
Epoch 5/20
1875/1875 [=====] - 2s 1ms/step - loss: 1.0156 - accuracy: 0.7485
Epoch 6/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.8506 - accuracy: 0.7928
Epoch 7/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.7360 - accuracy: 0.8208
Epoch 8/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.6501 - accuracy: 0.8418
Epoch 9/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.5827 - accuracy: 0.8585
Epoch 10/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.5288 - accuracy: 0.8706
```

SGD (Stochastic Gradient Descent)

- The actual gradient descent looks at the whole training data to update the weights. i.e the weights get updated once in every epoch
- SGD will change the weights for every sample record. Within an epoch the weights will get changed multiple times.
- SGD will do several iterations within an epoch of standard gradient decent

Mini-batch Gradient Descent

- GD updates weights over full training data
- SGD updates weights for every point
- Although SGD works, but in practise we don't want to update weights for every training sample.
- A single record used for updating weights will cause high variance in weights.
- May be it's a good idea to send batches of training samples to calculate the gradient
- We have more confidence while updating the weights based on a small subset of training rather than updating based on single sample

Mini-batch Gradient Descent

- Split the training dataset into small batches calculate the error, update the weights
- Most commonly used implementation of gradient descent
- Much faster than gradient descent and SGD
- This comes with another hyper parameter “batch size”
- Do not choose a very low number or really high number. The optimal batch sizes depends on your data.

LAB: GD, SGD and Mini-batch GD

- Data set : MNIST
- Model-1: Build a model using standard SGD - take a note of time taken and accuracy
- Model-2 : minibatch batch size 128- take a note of time taken and accuracy

Loss Functions

Loss Functions

Squared loss (E)

$$\frac{1}{2} \sum (y - \hat{y})^2,$$

Mean squared error (MSE)

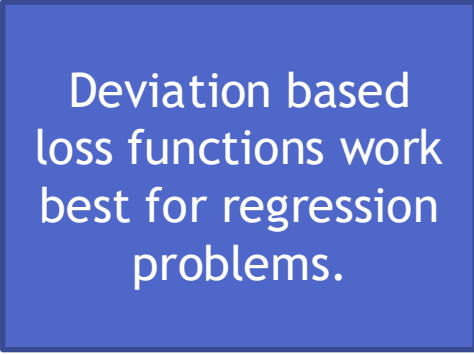
$$\frac{1}{n} \sum (y - \hat{y})^2$$

Mean absolute error (MAE)

$$\frac{1}{n} \sum |y - \hat{y}|$$

Mean absolute percentage error
(MAPE)

$$\frac{1}{n} \sum \frac{|y - \hat{y}|}{y}$$



Deviation based
loss functions work
best for regression
problems.

Cross Entropy

Binary cross entropy = $-\sum (y\log(\hat{y}) + (1 - y)\log(1 - \hat{y}))$

Actual Value (y)	Predicted Value ($\hat{y}$)	Cross-Entropy Formula	Loss (Log base-10)	Loss (Log base-e)
0	0.01	$-((0)\log(0.01) + (1)\log(0.99))$	0.004	0.01
0	0.99	$-((0)\log(0.99) + (1)\log(0.01))$	2	4.605
1	0.01	$-((1)\log(0.01) + (0)\log(0.99))$	2	4.605
1	0.99	$-((1)\log(0.99) + (0)\log(0.01))$	0.004	0.01

Cross Entropy

Cross – entropy for three classes = $-\sum (y_1 \log(\widehat{y}_1) + y_2 \log(\widehat{y}_2) + y_3 \log(\widehat{y}_3))$

Categorical cross – entropy = $\sum_{j=1}^m y_j \log(\widehat{y}_j)$ where $j = 1$ to m classes.

```
model.compile(loss='categorical_crossentropy', metrics=['accuracy'])
```

Cross entropy based loss functions
work best for classification
problems.

Conclusion

- We discussed regularization, activation functions, learning rate and optimization functions
- These hyperparameters finetuning decides the accuracy and efficiency of the model

Thank you
